

System Design Not For Interviews

Areg Melik-Adamyan, Petr Kurapov

April 2026

Contents

1	Preface; The Map and the Territory	4
2	Introduction to the Systems	8
2.1	Why a Chapter on What System Means	10
2.2	Four Traditions, One Claim	11
2.2.1	The engineering tradition: Crawley	11
2.2.2	The engineering tradition: NASA	11
2.2.3	The cybernetic tradition: Ashby	12
2.2.4	The biological tradition: Maturana and Varela	12
2.2.5	The philosophical tradition: Anscombe	13
2.2.6	The cognitive reading: Lakoff	13
2.3	The convergence	14
2.4	A Working Definition	14
2.4.1	Form and Function	15
2.5	Concept and Conceptual Integrity	16
2.6	The Four Tasks of System Thinking	18
2.6.1	Task 1: Identify the system, its form, and its function . .	18
2.6.2	Task 2: Identify the entities, the boundary, and the context	18
2.6.3	Task 3: Identify the relationships	19
2.6.4	Task 4: Identify the emergent properties	20
2.7	Complex vs. Complicated	21
2.8	Worked Example: Kubernetes as a System	21
2.9	What This Buys Us	22
2.10	Summary	23
3	Principles of Software Design	25
3.1	The Language of System Design	25
3.2	System Decomposition	25
3.3	Complexity and Its Causes	26
3.4	Design and Architecture	28
3.5	Architecture and Design Drivers	28
3.6	Software Architecture and Design	28
3.7	Requirements Definition	29

3.8	Examples of Poor Requirements	31
3.9	Good Requirements Examples	32
3.10	Conclusion	32
4	From Requirements to Conceptual Design	33
4.1	Introduction to System Design	33
4.2	From Requirements to Design	33
4.3	Phases in Systems Engineering	34
4.4	Conceptual Design	35
4.5	Understanding Concepts and Their Mapping	35
4.6	The Role of Functional Decomposition	36
4.7	The Importance of Mapping Functions to Forms	36
4.8	Designing for Flexibility: Refined Concepts and Variables	37
4.9	The Role of Iteration in Conceptual Design	38
4.10	Conclusion: The Art of Conceptual Design	38
5	System Decomposition and Modularity	39
5.1	The Importance of Cohesion and Well-Defined Interfaces	39
5.2	The Evolution of System Decomposition Techniques	40
5.3	Functional Decomposition and Its Limitations	40
5.4	The Problem with Over-Decomposition	41
5.5	Domain Decomposition as an Alternative	41
5.6	Object-Oriented Decomposition: A Modern Approach	42
5.7	Volatility-Based Decomposition: A Better Approach	43
5.8	System Decomposition in Practice	43
5.9	Abstraction through Categorization	44
5.9.1	Wittgenstein's Language Games	44
5.9.2	Cognitive categorization	47
5.10	Conclusion: Choosing the Right Decomposition Strategy	49
6	Abstraction and Module Design in System Architecture	51
6.1	Understanding Abstraction in System Design	51
6.2	The Concept of Deep Modules	51
6.3	Avoiding Shallow Abstractions	52
6.4	Designing Modularity: Generalization vs. Specialization	52
6.5	Managing Complexity Through Modularity	53
6.6	The Role of Layers in Deep Modules	53
6.7	Best Practices for Module Design	53
6.8	Conclusion	54
7	Practical Applications of System Design: A Case Study of Unix	55
7.1	Introduction to Unix and its Origins	55
7.2	The Unix Design Philosophy	55
7.3	Unix as a Modular and Composable System	56
7.4	Composition and Modularity in Unix: Filters and Pipelines	56
7.5	The Importance of Abstraction and Information Hiding	57

7.6	Lessons Learned from Unix Design	57
7.6.1	McIlroy's Principles	58
7.6.2	Rob Pike's Rules	58
7.6.3	Raymond's Rules (Selected)	58
7.7	Conclusion	59
8	Designing Build Systems and Automation Tools	60
8.1	What is a Build System?	60
8.2	Why Do We Need Build Tools?	60
8.3	Common Build Tools and Package Managers	60
8.4	The History of Make: The First Build Tool	61
8.5	Makefile Basics	61
8.6	Challenges in Using Make for Large Projects	62
8.7	Phony Targets and Clean-Up	62
8.8	Advanced Features of Make	62
8.9	Building a Custom Build System	63
8.10	Conclusion	63
9	From Algorithm to System Design: A Case Study of Google's PageRank	64
9.1	The Role of Algorithms in System Design	64
9.2	The Concept of the Web as a Large Graph	64
9.3	The Problem of Ranking in Web Search	64
9.4	PageRank: A Solution to the Ranking Problem	65
9.5	PageRank Algorithm and Its Mathematical Formulation	65
9.6	Challenges in Computing PageRank at Scale	65
9.7	Optimizing the PageRank Computation	66
9.8	Handling Dead Ends and Spider Traps in PageRank	66
9.9	Designing a System to Implement PageRank	66
9.10	The Role of Distributed Systems and MapReduce in Search Engine Design	67
9.11	Conclusion	67
10	Relational Model and Design of Modern Database Systems	68
10.1	The Early Days of Database Systems	68
10.2	Codd's Relational Model and Relational Algebra	68
10.3	Relational Algebra Operations	69
10.4	From Relational Algebra to SQL	69
10.5	Modern Database Systems and Query Optimization	70
10.6	Relational Algebra as a Theoretical Foundation for Query Execution	70
10.7	Conclusion	70

1 Preface; The Map and the Territory

Welcome to Software Design Not for Interviews.

Every textbook emerges from the author’s belief (sometimes hallucination) that something is definitely missing, explained wrong, or does not have enough depth. For two decades, during our work in industry and academia, we have consistently found dissatisfaction with poor design and the teaching of it, and an urge to explain how it should be done. The explanation part is tricky. Much harder than labeling someone a bad architect. Often, you realize that your counterpart has a different background, a different level of education, a different depth of thinking, and a different understanding of the field. They may also consider you a condescending jerk who wants to show off the title of the position in the code or documentation review. That makes it even harder.

This book exists because most “system design” content is optimized for performance in an interview, a very specific type of interviews, designing some Web grade request based system, that requires fast answers, neat boxes, memorized pattern matching and confident diagrams — while real software design, that you are going to participate is optimized for survival in a moving world: unclear requirements, partial information, competing incentives, legacy constraints, and teams that have to keep shipping while the ground shifts.

So, if you picked up this book hoping to memorize the “correct” design for a URL shortener or a chat application to pass a 45-minute whiteboard interview, please put it back on the shelf. There are plenty of resources for that game. We wrote this book for the work that happens after you get the job.

We are a collective of engineers, architects, and teachers who have spent decades wrestling with systems that refuse to fit into neat boxes. We have seen clean architectures rot because the abstractions didn’t match the human domain they served. We have seen “perfect” decompositions fail because they ignored the “grammar” of the underlying hardware.

In our industry, we often treat design as a purely technical endeavor—a matter of choosing the right database or the correct pattern from the Gang of Four. But as systems grow in complexity, technical prowess becomes secondary to cognitive clarity. How do we break a massive, amorphous problem into manageable pieces? How do we name those pieces so that their meaning survives when the original team leaves?

We believe that software design is the craft of decomposition and abstraction under volatility. You take a messy reality, decide what matters, or discover concepts that can carry meaning, and carve boundaries that let many people change many parts without breaking the whole in a systemic way. The primary outputs of design aren’t whiteboards or buzzwords — they’re interfaces, contracts, invariants, and categories that teams can coordinate around, comprising together that ephemeral structure — system.

We’ll make one claim again and again: A good decomposition is not “the correct structure.” It’s the structure that stays coherent as the system evolves.

That premise forces a different style of thinking than “find the right pattern.” It pushes you toward:

- naming as a design act (and renaming as maintenance),
- boundaries as agreements between communities,
- abstractions as useful conventions rather than eternal truths,
- and architecture as a map of practices, not a taxonomy of boxes.

To help with that, we borrow a few tools from outside typical software literature, especially philosophy of language and cognitive science — because, we believe they address the exact failure modes architects hit in practice: teams talking past each other, abstractions that “sound right” but don’t compose, and definitions that collapse under edge cases.

In Wittgenstein’s language games, meaning isn’t a hidden essence; it’s what competent participants do with words in a form of life. In architecture terms: an abstraction isn’t validated by its definition; it’s validated by how well it supports the system’s actual “game” — the workflows, invariants, and tradeoffs people rely on.

In Lakoff’s cognitive categorization, categories aren’t always crisp hierarchies; they are often prototype-based, fuzzy, and radial. In architecture terms: components and concepts often have “family resemblances” rather than one defining trait—and trying to force a rigid taxonomy can produce brittle design.

These are not academic detours. They’re practical lenses for:

- choosing boundaries without pretending they are eternal,
- designing APIs that match how people think and work,
- spotting conceptual mismatches early,
- and teaching teams a shared “grammar” for the system.

What might you get from this book? By the end, you should be able to: start from messy requirements and extract stable concepts and invariants, decompose a system by identifying volatility and locating seams, design interfaces as rules of play (not “bag of methods”), build an internal vocabulary that reduces coordination cost, and evaluate design choices with explicit tradeoffs and failure modes.

This book is written for engineers in the trenches, who build systems that outlive their initial team: staff+ engineers, tech leads, architects, engineering managers, and students who want to learn design as a real craft rather than a performance.

In this book, you will find fewer mentions of “Microservices vs. Monoliths” and more discussions on how Ludwig Wittgenstein’s philosophy of language can save you from bad API design. You will see how George Lakoff’s cognitive theories on categorization can help you define data structures that feel natural rather than alien. We treat the System Architect not as a dictator of rules, but as a cartographer mapping a complex terrain.

We will walk you through the lifecycle of engineering thought—from the first vague requirement to the concrete reality of a running system. We will dismantle the "Interview Design" mindset, which favors rote memorization, and replace it with "Engineering Design," which favors context, volatility analysis, and evolutionary stability.

Finally, we ground these high-minded concepts in the dirt and grime of reality. We will deconstruct the systems you use every day—Git, Make, Google Search, PyTorch—to show you that the "magic" inside them is just rigorous decomposition applied with intent.

This is not a book of answers. It is a book of better questions.

Roadmap: A Syllabus for the Architect This book is structured to guide you from the abstract philosophy of design to the concrete application of those principles in world-class software systems.

Chapter 1: The Architect's Burden The "Why" and the Context. We start by deconstructing the modern software crisis. We explain why "LeetCode architecture"—optimizing for the interview—creates brittle systems in production. We introduce the role of the architect not as a code-writer, but as a manager of complexity and a reducer of cognitive load. We define the difference between "complicated" and "complex" systems and set the stage for design as a survival mechanism.

Chapter 2: First Principles of Software Design The Physics of Engineering. Before we build, we must understand the laws that govern us. This chapter covers the invariant principles that remain true regardless of the tech stack: Coupling and Cohesion, the Fallacy of DRY (Don't Repeat Yourself), the concept of Hysteresis in software evolution, and the fundamental trade-off between Efficiency and Resilience.

Chapter 3: Requirements and "Forms of Life" Discovery Phase. Software models a domain. If the map doesn't match the territory, the system fails. We move beyond "User Stories" to discuss requirements gathering through the lens of Wittgenstein's "Forms of Life." We learn to identify the grammar of a business domain—the unwritten rules and social practices that the software must support, rather than contradict.

Chapter 4: Concepts and Conceptual Design The Mental Model. How do we turn a requirement into a concept? This chapter focuses on the Pre-Code phase. We discuss how to invent concepts that stick. We explore the "System Metaphor" (is this a pipeline? a marketplace? a nervous system?) and how early naming decisions dictate the long-term fate of the architecture.

Chapter 5: System Decomposition strategies The Art of Breaking Things Apart. The heart of system design. We critique standard decomposition methods (functional, layered) and introduce Volatility-Based Decomposition. We learn to separate things that change at different rates and for different reasons. This is where we learn to isolate the "hard" core of the system from the "soft" periphery.

Chapter 6: Modularity and Interface Design The Boundaries. Once the system is broken down, how do the pieces talk? We treat APIs not just as function calls, but as contracts and language games. We discuss the "grammar"

of interfaces—synchronous vs. asynchronous, stateful vs. stateless—and how to prevent “grammar leakage” where the implementation details of one module infect the interface of another.

Chapter 7: Abstraction and Categorization The Cognitive Layer. Featuring the section on Wittgenstein and Lakoff. Here we dive deep into the theory of abstraction. We reject the Aristotelian view of rigid categories in favor of Prototype Theory and Radial Categories. We explore how to design “fuzzy” abstractions that can evolve. We discuss how to map the “Family Resemblances” of system components to create polymorphic interfaces that actually make sense to human developers.

Chapter 8: Practical Application I — The Deterministic World Case Study: Build Systems and Version Control. We apply our decomposition rules to systems that demand absolute precision.

Chapter 9: Practical Application II Git - We analyze how Linus Torvalds decomposed “history” into a Directed Acyclic Graph (DAG) of content-addressable storage. Build Systems (Make/Bazel): We look at how “dependencies” are abstracted and how the graph metaphor drives reproducible builds.

Chapter 10: Practical Application III — The Probabilistic World Case Study: Search and Ranking. We move to systems where answers are “maybe” rather than “yes/no.”

Chapter 11: PageRank: We explore how the web was decomposed into a matrix of eigenvectors. We look at the abstraction of “Authority” and how a mathematical insight became a system architecture. Here we show how the decomposition can work for scale when the dataset is the entire world.

Chapter 12: Practical Application III — The Learning World Case Study: Deep Learning Systems. We conclude with the most complex systems of the modern era. Deep Learning: We deconstruct the architecture of frameworks like PyTorch and TensorFlow. We analyze the “Tensor” as a radial category (math array + memory view + accelerator payload). Backpropagation: We look at how “learning” is systematized into the abstraction of Automatic Differentiation (Autograd) and the computational graph.

A note on style

We’ll be plainspoken. We’ll prefer falsifiable statements to vibes. We’ll name the tradeoffs we’re making. And we’ll repeatedly ask a question that interview prep rarely asks:

“What will change in 5 years when the company is sold to a competitor, most of the team is laid off, and where do we want that change to be cheap?”

2 Introduction to the Systems

System design takes a central role as software development matures and productivity accelerates. Unlike many traditional engineering disciplines, where natural constraints like physical laws and material properties sharply limit design choices, software doesn't have a strict definition. It exists in a virtually boundless design space. With minimal hard constraints beyond compute, bandwidth, and memory, we are free to create nearly anything. But with that freedom comes a daunting cost: complexity.

Complexity in software is self-propagating. Every poorly framed abstraction, leaky interface, or ambiguous responsibility adds friction to reasoning, extending development time and increasing the likelihood of bugs. Over time, we observe a familiar pattern across the industry: systems become harder to understand, modify, scale and reason about. Software quality degrades not because we lack effort or tools, but because we lose control over complexity.

The most powerful tool we have to fight complexity is high-quality design. But teaching and learning how to design large, complex software systems remains one of the most underdeveloped areas in our field. While over the past several decades we have invented robust tools, methodologies, and frameworks starting from optimizing compilers, "agile" processes, software patterns, layered architectures, etc, the underlying skill of design itself often remains elusive, intuitive, or reserved for the most experienced. There are very few texts that address these issues, and even fewer that teach that.

This book aims to change that.

We take a focused stance: a several-decades combined experience taught us that decomposition and composition are the most challenging parts and the essence of software design. These two interlocking forces – how we break systems down, and how we build them back up – form the core of every design decision. Our goal is to help you understand these forces deeply, master the techniques that govern them, and develop the judgment required to wield them well.

Rather than offer a collection of patterns or interview tricks, this book builds a language and a way of thinking. It teaches software design not as a checklist, but as a practice, as any good philosophy book does.

We organize the book around the following key topics:

The Language of Design: The terms, mental models, and conceptual frameworks that allow us to think clearly about system architecture and structure.

System Decomposition: How to break a complex system into components that are cohesive, loosely coupled, and individually understandable.

Abstraction: Techniques for hiding irrelevant details and shaping interfaces that enable comprehension, substitution, and scalability.

Creation and Synthesis: How to construct systems from parts, ensuring that what is assembled is more than just the sum of its components.

Building Systems: Practical strategies for constructing scalable, maintainable, and evolvable software that stands the test of time.

We stand on the shoulders of giants: Fred Brooks' reflections on software complexity and conceptual integrity [Bro10], Richard Gabriel's explorations of

design and aesthetics [FG10], John Ousterhout’s emphasis on simplicity [Ous18], Eberhardt Rechtin’s systems thinking, and the applied guidance from Cervantes and Kazman [CK16] and Lowy [Low19]. These thinkers have shaped our understanding of what good design means and why it matters.

This book is for practitioners who are building real systems, not solving toy problems. It is for those who want to understand why systems fail as they grow, and what it takes to build ones that do not. If you’re tired of interview-oriented design advice and want to master the foundational thinking behind large-scale software systems, you’re in the right place.

In the first chapter we will talk about The Language of Design. Before we can design systems well, we must learn to speak about them precisely. The practice of software design, like all disciplines, is grounded in language. Without a shared vocabulary, teams cannot reason about structure, discuss tradeoffs, or critique architectures. The words we choose affect what we see—and what we are blind to.

Most software teams suffer not from a lack of intelligence or good intentions, but from a lack of shared concepts. We confuse implementation with architecture, code with system, modules with services, and so on. If we cannot distinguish between cohesion and coupling, between interface and boundary, or between layering and dependency, our designs degrade into vague shapes held together by assumptions and tribal knowledge.

This chapter introduces a foundational vocabulary for design. Our aim is not to be pedantic, but to clarify and empower. Like any craft, software architecture benefits from precise terms that map to concrete experiences.

We will explore the following core concepts:

- System and Subsystem,
- Component, Module, and Service,
- Interface and Boundary,
- Layer, Dependency and hierarchy,
- Invariants and Contracts.

This language is not just for architects. It should permeate the entire team – from junior engineers to technical leads. Precision in terminology supports precision in thought and clarity in design. Every programmer is familiar with the Gang of Four books on Design Patterns and knows that it was

“You cannot change what you cannot name.” – Systems Thinking
Principle

System design takes a central role as productivity increases. Design space for software is colossal in comparison to other engineering disciplines due to limited restrictions (only bandwidth, capacity, compute). Complexity spawns complexity. We observe an overall software quality degradation. The most powerful tool to fight complexity we have is high-quality design. Yet, teaching how

to design large, complex systems is non-trivial. Over the last several decades, the field has developed tools and methods for building better systems. Yet, the art of design remains elusive. In this book, we focus on the two (and only) main aspects of system design: decomposition and composition.

The book covers the following key topics:

- **Language of the Design:** The terminology and conceptual frameworks used in system design.
- **System Decomposition:** Techniques for breaking down complex systems into manageable components.
- **Abstraction:** Creating simplified models that hide unnecessary complexity.
- **Creation and Synthesis:** The process of putting together the various elements of a system to create a cohesive whole.
- **Building Systems:** Practical approaches to building scalable, maintainable software systems.

Ousterhout [Ous18], Fairbanks [FG10], Brooks [Bro10], Lowy [Low19], Cervantes and Kazman [CK16].

2.1 Why a Chapter on What System Means

On September 23, 1999, the Mars Climate Orbiter arrived at Mars and was never heard from again. The investigation produced one of the most cited single-cause explanations in engineering history: one software component produced thrust values in pound-seconds, another consumed them as newton-seconds, and the spacecraft descended too low, disintegrated in the atmosphere, and cost NASA \$327.6 million.¹

That explanation is true. It is also misleading. No line of code was wrong. Every module met its specification. Every engineer did competent work. The engineer who wrote the producer did not know whose code would consume the output three years later and had no occasion to check. The reviewers of each module verified the module. The integration testers verified the integrated behavior *in simulation*, which used the producer’s output directly and never exercised the unit conversion path under flight conditions. The failure was not in any part. It was in the relationships between parts. It was in the *system*.

This is the first thing to understand about systems: the questions “who was wrong?” and “what was wrong?” can both have answers of “nothing,” and the system can still fail. Failure, like success, is a property that lives in

¹Mars Climate Orbiter Mishap Investigation Board, *Phase I Report*, NASA, November 10, 1999. The proximate cause was that ground software produced impulse in pound-force seconds while the orbiter software expected newton-seconds; over the nine-month cruise the accumulated trajectory error placed the spacecraft roughly 100 km lower than intended at Mars arrival. The relevant point for our purposes is not the unit error but the organizational structure that allowed an interface assumption to go unverified.

the connective tissue—in the pattern of interactions, the pattern of assumptions, and the pattern of what was not asked. You cannot find it by looking inside any one component, any more than you can find a conversation by examining the two people involved in it individually.

This is also why we need a chapter on what “system” *means*. The word is unfortunately flexible—everything from a solar system to a filing system to an immune system gets labeled one—and this flexibility lets bad designers hide. If everything is a system, then calling something a system tells us nothing. We need the word to do work. What we want is a definition sharp enough that you could look at a proposal and say “this isn’t actually a system yet, it’s a heap.”

A note on method. We are not going to chase an essence. As Wittgenstein pointed out, asking “what is the essence of X?” is often a way to burn a conversation rather than clarify it, and as Lakoff’s cognitive work shows, most useful concepts are not essence-defined but prototype-organized. We will instead look at four independent traditions that had to grapple with what a system is—engineering (Crawley, NASA), cybernetics (Ashby), biology (Maturana and Varela), and philosophy of action (Anscombe)—and observe that they converge. Where four traditions starting from unrelated motivations converge on the same structural claim, we should pay attention.

2.2 Four Traditions, One Claim

2.2.1 The engineering tradition: Crawley

Crawley’s definition is the operational one [crawley2016]:

A system is a set of entities and their relationships, whose functionality is greater than the sum of the individual entities.

Two claims are doing work here. First, a system has *entities* and *relationships*—not just parts, but parts connected in specific ways. Second, when those entities interact through those relationships, a *function* appears that none of the entities has on its own. This second property is called **emergence**, and it is the reason we build systems in the first place.

The Crawley test: *a brick is not a system; a brick wall is*. A single brick is uniform throughout. A brick wall has entities (bricks, mortar) and relationships (geometric arrangement, load transfer) that together produce a function (holding up a roof) that neither bricks nor mortar produce individually.

2.2.2 The engineering tradition: NASA

The NASA Systems Engineering Handbook, working independently of Crawley, offers [nasase2016]:

A system is a construct or collection of different elements that together produce results not obtainable by the elements alone.

The vocabulary differs (“construct,” “elements,” “results”) but the structure is identical: a distinguished whole, composed of interacting parts, exhibiting effects irreducible to the parts. NASA additionally emphasizes that the elements are *deliberately selected* and the results are *intended*—which matters for us, because we are not studying solar systems; we are building things on purpose.

2.2.3 The cybernetic tradition: Ashby

W. Ross Ashby, working on problems of regulation and control, defined a system as any collection of variables that an observer chooses to study, together with the transitions between their joint states [ashby1956]. Stripped down, this is:

- A **state space**: the set of all possible configurations the system could be in.
- A **transition structure**: the rules by which the current state leads to the next.

Ashby’s crucial contribution is the **Law of Requisite Variety**: to regulate a system subject to disturbances, the regulator must have at least as much *variety* (distinguishable states) as the disturbances it must handle. A thermostat that knows only “above setpoint / below setpoint” cannot regulate a furnace whose real disturbance space has three modes (cold, warm, condensing-humidity); it will behave correctly two-thirds of the time and fail in the third without anyone being “at fault.”

Ashby tells us something Crawley and NASA don’t: the complexity of a system is not a budget line item that can be cut to taste. It is bounded below by the variety of the problem. You can move complexity, you can hide it, you can push it down or across—but if you remove it outright, the system will fail in the exact proportion by which the remainder is insufficient.

2.2.4 The biological tradition: Maturana and Varela

Maturana and Varela, studying what distinguishes a living thing from a non-living one, arrived at the concept of **autopoiesis**: a system whose primary activity is the continuous production and maintenance of the very components (and the boundary) that define it [maturana1980]. A cell is not identified by the particular atoms currently inside it—those turn over constantly. It is identified by the network of processes that keeps producing a cell-shaped arrangement of the right kinds of components, and by the membrane that keeps distinguishing inside from outside.

This matters for software because **our most reliable production systems are autopoietic in exactly this sense**. Consider Kubernetes. If you ask “what is the deployment?”, the correct answer is not “these three pods running on these three nodes.” Those pods may be killed and replaced within seconds. The deployment is the control loop: a declared desired state, a continuous observation of actual state, and a corrective action that drives actual toward

desired. Components churn; the system persists. Kill a pod, the controller spawns a replacement. Drain a node, the controller reschedules. The identity of the system is in the loop, not in any snapshot.

Maturana and Varela tell us something neither Crawley nor Ashby makes explicit: for long-lived systems, **structure is a verb, not a noun**. The system is the pattern of self-maintenance.

2.2.5 The philosophical tradition: Anscombe

Elizabeth Anscombe, in *Intention* (1957), analyzed what makes an event an *action* rather than a mere happening. Her answer—radically compressed—is that an event is an intentional action when it is intelligible under a chain of “why” questions, each answer pointing to a reason the agent endorses [anscombe1957]. “Why are you pumping?”—“To fill the cistern.”—“Why fill the cistern?”—“To supply the house with water.”—“Why supply the house?”—“Because the people in it need water to live.” The chain bottoms out in something the agent takes as a reason, full stop.

Transported into engineering, Anscombe’s why-chain is the structure of **function**. Any part of a designed system ought to answer some why-question, and that answer ought to connect to the next level up. If a module exists and no one can say what higher-level function it serves, it is either (a) dead code, (b) a historical accident that survived a refactor, or (c) a load-bearing thing nobody understands—all three are dangerous, and which of the three it is, is itself worth finding out.

Anscombe also gives us a diagnostic for requirements. A requirement that cannot be traced upward to a stakeholder concern (“why does this requirement exist?”) is either noise or it is legitimate and someone has failed to write down the concern that justifies it. Either way it is not finished work.

2.2.6 The cognitive reading: Lakoff

A final perspective worth naming is George Lakoff’s, and it is pitched at a different level than the four above. Where Crawley, Ashby, Maturana, and Anscombe each offer a definition of system, Lakoff’s work on prototype theory and conceptual metaphor offers a theory of why their definitions overlap the way they do—and why our working definition in the next section will have five criteria rather than one [lakoff1987; lakoff1980]. Lakoff observed that most of our operative concepts—“bird,” “game,” “mother,” “system”—are not defined by necessary and sufficient conditions. They are organized around *prototypes* (a robin is a more typical bird than a penguin) with radial extensions linked to the center by metaphor, function, or family resemblance. What we mean by “system” across engineering, cybernetics, biology, and philosophy is one such radial category: the prototype is something like “a set of interacting things that together do something no one thing could do alone,” and the four traditions are radiating out from that prototype along different axes—structural for Crawley, regulatory for Ashby, processual for Maturana, intentional for Anscombe. The

convergence is not coincidence. It is what it looks like when four communities independently circle the same prototype and each notice a different facet. Two operational consequences follow. First, our working definition (§2.4) will not be wrong for having five criteria rather than one clean necessary-and-sufficient condition; it will be honest. Second—and more consequential for everyday architectural work—when we design a system we are choosing metaphors as well as structures. “Pipeline” invites linearity and bottleneck analysis. “Mesh” invites parallelism and partial failure. “Tree” invites traversal and ownership. “Market” invites contracts and incentives. “Organism” invites homeostasis and repair. The metaphor is not decoration on top of the architecture; it is part of the architecture, because it silently rules out some decompositions and makes others feel inevitable. An architect who does not notice their own metaphors is an architect whose options have already been chosen for them.

2.3 The convergence

These four traditions were written by people who had no reason to be reading each other. They converge on a structure:

Tradition	What it emphasizes	What it adds
Crawley / NASA	Entities, relationships, emergent function	Operational vocabulary for building
Ashby	State space, variety, regulation	A <i>law</i> relating complexity to environment
Maturana / Varela	Self-maintaining closure, boundary	Identity through process, not substance
Anscombe	Why-chains, intentional structure	Function as answerable reason

Table 1: Four traditions converging on the concept of system.

All four insist that a system is not its parts, that identity is carried by relations or processes, and that the behavior of the whole is the primary object of study. Where they disagree is on which aspect is fundamental—and for our purposes we do not need to choose. Engineering systems are simultaneously entity-networks (Crawley), variety-matchers (Ashby), self-maintaining loops (Maturana), and intentional structures (Anscombe). A good definition should accommodate all four.

2.4 A Working Definition

Taking the convergence seriously:

A system is an interacting collection of elements, bounded from its context, whose relationships sustain a function that no element realizes alone, and that function exists to answer a “why” that a stakeholder endorses.

Five pieces, none removable:

1. **Interacting collection of elements.** Not a heap. The elements must be doing something to each other.
2. **Bounded from its context.** Someone drew a line. The line is a decision, and the decision must be defensible when we discuss requirements.
3. **Relationships that sustain function.** Both formal structure (how things are arranged) and functional interaction (what flows between them), in Crawley’s vocabulary.
4. **Function not realizable by any element alone.** The emergence test. If one part could do the whole job, you don’t have a system, you have redundant packaging.
5. **Function exists to answer a why.** The Anscombe anchor. Without this we cannot tell a good system from a merely working one.

A filing cabinet full of arbitrary documents is not a system by this definition: (1) passes, (2) passes, (3) and (4) fail—arranging documents alphabetically is not the same as the arrangement *doing work*. A cell is a system: autopoietic closure supplies (1)–(4), and (5) is supplied by evolution in the descriptive case or by us when we use cells as components. Kubernetes is a system on all five counts, and we will return to it repeatedly in later chapters.

2.4.1 Form and Function

Having a definition is not yet having a method. Crawley’s operational contribution is a pair of concepts sharp enough to actually use: form and function.

Form is what the system *is*—its physical or informational embodiment. Form is static on the timescale we care about. It has shape, layout, structure. You can point to it. The heart *is* a four-chambered muscular organ with specific geometry. A Kubernetes cluster *is* a set of nodes, a control plane, an etcd store, a set of deployed pods.

Function is what the system *does*—the activity, operation, transformation. Function is action. It is not form. Critically, in Crawley’s framing, function always has two parts:

function = process + operand

The process is the verb; the operand is the thing the verb acts on. “Pumps blood.” “Amplifies voltage.” “Reconciles desired state.” “Routes requests to healthy backends.” Every function in a well-specified system can be written in this form, and when it cannot, you usually have found a place where thinking is muddy.

This matters operationally because it gives us a regular shape for requirements and a regular shape for the Anscombe why-chain. When “why?” is asked

of a process+operand pair, the answer is a higher-level process+operand pair. “Reconciles desired state”—why?—“maintains service availability.” “Maintains service availability”—why?—“upholds the SLA with customer.” At the top of the chain is the stakeholder’s reason, which terminates the regress.

A system’s design is, in large part, a *mapping* from functions to forms. The same function can be realized by different forms (an amplifier circuit and a software equalizer both amplify signals), and the same form can enable different functions (a database stores records today and provides an audit log tomorrow). Requirements live in the function domain. Designs live in the mapping. Implementation lives in the form. Confusing the three is the most common beginner error in architectural reasoning, and it remains common in practitioners well past beginner. The mapping itself is a structural object worth naming on its own, and the next section does so.

2.5 Concept and Conceptual Integrity

A system has form and function. Between them sits a third structural element that neither of the first two contains: the **concept**—the organizing idea under which a given form delivers a given function. In Crawley’s vocabulary, a concept is the abstraction that links form to function and makes the linkage intelligible. Git’s concept is “a content-addressable versioned filesystem.” Unix’s concept is “everything is a file.” A refrigerator’s concept is “a box that maintains a cold environment to slow spoilage.” None of these is a form (the blob, the file descriptor, the compressor) or a function (tracking versions, uniform I/O, preserving food). Each is the idea that lets a particular form make sense of a particular function. When a single concept rules an entire system we say the system has **conceptual integrity**, and this section argues that concept and integrity are the backbone on which the rest of the architectural apparatus hangs.

Concepts are not unique. The function “preserve food” admits several concepts—chilling, drying, vacuum-sealing, fermenting, irradiating—each mapping to very different forms and carrying different trade-offs. A refrigerator, a dehydrator, and a vacuum-sealer all perform the same function, but they embody different concepts, and those concepts are where the engineering choices live. The concept you choose is the first real design decision, in the sense that every subsequent decision about form must fit it.

What makes concepts load-bearing is that a good concept does not map one function to one form; it maps *many* functions to a coherent family of forms under a single rule. Unix’s “everything is a file” maps **open**, **read**, **write**, **seek**, and **close** onto regular files, pipes, devices, sockets, **/proc**, **/sys**, and network streams under one dispatch mechanism. Git’s content-addressable-object-DAG maps branching, merging, tagging, stashing, cherry-picking, and rebasing onto blobs, trees, commits, and references - all by hash. In categorical terms, a good concept behaves close to a *functor*: composition on the function side maps to composition on the form side, regularly enough that reasoning about one side tells you what to expect on the other.

This brings us to Brooks’ most cited claim about system design, from *The Mythical Man-Month* and developed in *The Design of Design* [brooks1975; brooks2010]:

Conceptual integrity is the most important consideration in system design. It is better to have a system omit certain anomalous features and improvements, but to reflect one set of design ideas, than to have one that contains many good but independent and uncoordinated ideas.

The claim lands harder once you have the form-function-concept vocabulary. Conceptual integrity is the demand that a system be ruled by *one* concept, not many—that the same organizing idea covers every part, so that someone who understands one corner of the system can predict how an unfamiliar corner behaves by analogy to the concept, not by memorizing special cases. Unix has conceptual integrity because `cat /dev/random | grep -v 0 > file` is intelligible without inventing a new mental model for any of the pieces. A system without conceptual integrity makes its users and its maintainers carry a separate mental model for each feature, and the 7 ± 2 ceiling on working memory turns every integration task into a paging problem.

What conceptual integrity buys, concretely, is a *principle of selection* for new features. When a new requirement arrives, the question is not “can we build this?” but “does this fit the concept?” If yes, the extension is radial in Lakoff’s sense—it fits the prototype, composes with what is already there, and the mental model does not fracture. If no, the choice is to reject the feature, reshape it until it fits, or admit that the concept itself needs to change—the last of which is expensive, conscious work rather than drift. This is Brooks’ point about the Reims Cathedral: built across centuries by changing hands, it still reads as one building, while a system “designed by committee” reads as a collision of voices even when each voice was individually reasonable.

Two connections to earlier material are worth naming. First, the concept is the horizontal axis of coherence; Anscombe’s why-chain is the vertical axis. The why-chain says every function traces upward to a stakeholder reason; conceptual integrity says every function at the same level traces across to the same organizing idea. A system with both can be reasoned about in two directions—up the why-chain and sideways across the concept—and both directions yield the same model. Second, the concept is precisely the Lakoff prototype. Conceptual integrity means extensions of the system are radial extensions of the prototype, related to the center by metaphor, function, or family resemblance rather than by arbitrary resemblance. “Everything is a file” makes a socket a radial extension of a regular file. “Sockets happen to have an fd” makes it an arbitrary resemblance. The first preserves integrity; the second slowly erodes it.

The failure mode to watch for is what Brooks called the system “designed by committee”: many good but uncoordinated ideas, each locally sensible, aggregating into a whole whose mental model is a list rather than a principle. The Mars Climate Orbiter with which this chapter opened is a failure of conceptual integrity at the interface layer. The concept of “what passes across this

boundary” was never made explicit and unified, so one side carried one concept (English units of impulse) and the other carried another (SI units), and the seam failed silently. A system with integrity would have had a concept at the interface—*interfaces carry dimensioned quantities, not raw numbers*—and the failure would have been caught at design review because it would have been a concept violation, not a numerical one.

Maintaining conceptual integrity across a large, long-lived system is the most expensive discipline in architectural practice. It is also the load-bearing one. Without a single ruling concept the other practices—deep modules, volatility-based decomposition, grammar-respecting boundaries—have nothing to be disciplined *against*, and they degrade into local tidiness on top of global incoherence. The concept is the backbone on which integrity hangs, and integrity is what makes the rest of the chapter’s apparatus add up to a designable system rather than a clever collection of isolated techniques.

2.6 The Four Tasks of System Thinking

With definition and vocabulary in place, we can state what it means to *think* about something as a system. Following Crawley, there are four tasks, and they are carried out in a loop rather than a sequence—later tasks feed back into earlier ones.

2.6.1 Task 1: Identify the system, its form, and its function

State what the system *is* and what it *does*, in that order, concisely. If you cannot write both in a paragraph, you have not yet understood the system well enough to design it.

Example. System: the Linux Virtual File System (VFS). Form: a kernel-resident dispatch layer that presents uniform file operation types (**open**, **read**, **write**, **stat**, **mmap**, ...) and delegates each operation, by inode metadata, to an installed filesystem driver (ext4, btrfs, NFS, procfs, ...). Function: presents the process “access named persistent data” operating on operand “file-like object” uniformly across heterogeneous storage backends.

Counterexample. “It’s a microservice for handling user stuff” is not an answer to Task 1. Neither is “it’s the order service.” The second is a name, not a description; the first is a slogan.

2.6.2 Task 2: Identify the entities, the boundary, and the context

Decompose the form. Decide what is in and what is out. This is never innocent: where you draw the boundary determines what you will see, what you will be responsible for, and what you will treat as given.

Crawley identifies five sub-operations within this task. They are worth naming because in practice they are conflated and each failure mode has its own signature:

1. **Initial decomposition.** Discrete systems (a fleet, a team) are trivial to decompose. Modular systems (an amplifier, a well-factored codebase) are straightforward. *Integral* systems—steering mechanisms, analog circuits, tangled legacy software—cannot be cleanly decomposed without re-architecting them, and pretending otherwise is how integration disasters happen.
2. **Holism.** Expand the list. Who *might* matter to this system? Saarinen’s principle: design the chair in the room, the room in the house, the house in the city. In software: the deployment in the cluster, the cluster in the org, the org in the regulatory environment.
3. **Focus.** Contract the list. Humans can reason about roughly 7 ± 2 things simultaneously. The rest must be set aside *knowing they are set aside*, not forgotten.
4. **Abstraction.** Collapse details. Two analysts become “Analytics.” Three regional load balancers become “Ingress.” Abstractions must preserve what matters and hide what doesn’t; this is a judgment call and a major source of error.
5. **Boundary.** Draw the line. What is in is your responsibility. What is out is your context. The boundary often has legal, contractual, or organizational force beyond the engineering reasoning, and those forces must be acknowledged.

2.6.3 Task 3: Identify the relationships

List the connections between entities. Crawley distinguishes two kinds and insists on both:

- **Formal relationships**—the structure. A is connected to B. A contains B. A has a network route to B. The Linux VFS is *linked* to the ext4 driver via a vtable of function pointers. These are the static, enduring connections.
- **Functional relationships**—the interactions. A sends requests to B. A pumps blood to B. The VFS *dispatches* `open()` calls to ext4. These are dynamic, transient, and they are what actually delivers the system’s function.

A functional relationship requires a formal relationship—the heart cannot pump blood to the lungs without a pulmonary artery connecting them—but the formal relationship does not guarantee the functional one. A network route exists; whether the message gets through depends on policy, throttling, authentication, serialization, and a dozen other things along the way. The Mars Climate Orbiter had the formal relationship (the consumer did consume the producer’s output). The functional relationship was broken by an unexpressed assumption about units.

Relationships that cross the system boundary are **external interfaces**. These deserve special attention because they are where the system meets what we do not control. Most integration failures live at external interfaces, and most architectural decisions with long half-lives are decisions about external interfaces.

2.6.4 Task 4: Identify the emergent properties

Predict what the system will *do* when its entities, with their functions, are wired together by their relationships. This is the hard task, and Crawley catalogs emergence along two axes:

	Anticipated	Unanticipated
Desirable	Cars transport people	Cars create a sense of freedom
Undesirable	Cars emit CO ₂	Cars kill people

Table 2: Types of emergence, after Crawley.

Our work as designers concerns primarily the top-left (make the good thing happen) and the bottom-right (don't let the bad thing happen). The bottom-right is the **emergency**—literally, something that *emerges* under conditions nobody modeled. The A320 Warsaw crash in 1993 is a canonical example: the software prevented deploying the thrust reverser until both landing gear struts were compressed, because a reverser deployed mid-air is catastrophic. In a cross-wind, one wing was held low, one strut did not compress, the reverser did not deploy, and the aircraft overran the runway. Every component behaved exactly as specified. The emergency emerged from the interaction of specifications with a scenario no one in specification review had pictured.

There are four ways to predict emergence, in decreasing order of reliability:

1. **Precedent.** You (or someone you trust) has built this exact thing before, and it emerged as expected. This is why experience is expensive and valuable.
2. **Experimentation.** Build it, or part of it, and see what happens. Spiral development and chaos engineering are variants of this.
3. **Modeling.** If the functional behavior of the parts can be formalized and the relationships are well understood, the whole can sometimes be predicted. This is why integrated circuits with billions of gates work: the transistor model is tight and the composition rules are mathematical.
4. **Reasoning.** For unprecedented, un-experimentable, un-modelable systems, we are left to think carefully. This is the residual, and it is where architectural judgment lives.

Ashby’s Law bites hardest here. If the entities and relationships do not between them carry enough variety to cover the real problem, some disturbance will find the gap and produce an emergency. You can sometimes predict *that* this will happen without being able to predict *which* disturbance will do it. That is a useful class of prediction; we will develop it in Chapters ?? and ??.

2.7 Complex vs. Complicated

A distinction that will recur throughout the book:

- **Complex** is a property of the system. A system is complex when it has many entities and many relationships, so that describing it requires a lot of information. Crawley measures it roughly as layers of decomposition required: simple (one level, $\leq 7 \pm 2$ entities), medium (two levels, ≤ 80 -ish), complex (three or more, with abstractions of abstractions).
- **Complicated** is a property of our *ability to understand* the system. A system is complicated when its apparent complexity exceeds our capacity to reason about it. The same system can be complex but not complicated (a well-architected Linux kernel) or simple but complicated (a poorly-written 500-line shell script).

Ashby’s Law gives the lower bound on necessary complexity: it has to carry the variety of the problem. Architectural craft consists of making systems that have the complexity the problem requires but are *not* complicated for the people who must build, operate, and evolve them. This is why we decompose, abstract, layer, and insist on deep modules—not to reduce complexity, which is bounded below by the problem, but to manage the *apparent* complexity that any given mind must hold at once.

The one-sentence job description of the system architect, which we borrow from Crawley:

Build systems of the necessary complexity that are not complicated.

2.8 Worked Example: Kubernetes as a System

To illustrate the framework converging on a concrete artifact, consider Kubernetes.

System / form / function. Form: a distributed control-plane process (API server, scheduler, controller-manager, etcd) plus a per-node agent (kubelet) plus a CNI network plane plus declarative configuration artifacts. Function: accepts a declaration of desired state (Deployments, Services, ConfigMaps) and, via continuous reconciliation, produces an observed cluster state that matches, operating on the operand “containerized workloads.”

Entities, boundary, context. Entities inside: API server, etcd, scheduler, controllers, kubelet, container runtime, CNI. Boundary: the cluster’s external

API surface plus the node boundary. Context: the underlying cloud or hardware, the container registries, the human operators, the applications running inside. Holism reminds us that the runbook, the alert rules, the on-call roster, and the finance team that sets the compute budget are all in the larger system; focus keeps us on the reconciliation engine today.

Relationships. Formal: kubelet watches the API server; controllers subscribe to resource changes via watch streams; etcd stores all authoritative state. Functional: desired-state objects flow in; status updates flow back; scheduler issues bind decisions; kubelet issues container lifecycle operations. The functional pattern is everywhere the same shape: *observe* $\rightarrow$ *diff against desired* $\rightarrow$ *act* $\rightarrow$ *record outcome*.

Emergence. The emergent property is **self-healing**: the cluster maintains declared workload state despite arbitrary component failures, short of total loss of the control plane. No single component realizes this. The scheduler alone cannot heal a failed pod. The kubelet alone cannot decide where to place a replacement. etcd alone cannot observe the world. Self-healing emerges from the reconciliation loops operating over the shared declarative state.

Autopoietic reading. The pods you are running right now will very likely not be the pods running in an hour. The cluster persists. What persists is the loop, not the contents. This is Maturana and Varela exactly: identity is in the process that produces and reproduces the boundary and the components, not in the components themselves.

Ashby reading. The environment Kubernetes regulates is a high-variety one: machines fail, networks partition, processes leak, loads spike, config changes land. The reconciliation control loop has sufficient variety to respond to this disturbance space because it operates on a small number of primitives (observe, diff, act) composed uniformly across every resource type. If you attempted to regulate Kubernetes's problem space with fewer primitives, some disturbance would find the gap; if you tried to regulate it by enumerating responses to each disturbance, the complexity would explode.

Anscombe reading. Why does the scheduler exist? To place pods on nodes. Why? To realize declared workload locations without operators picking nodes. Why? To let application teams declare intent without operational detail. Why? To let the organization ship software at the rate the market demands. The chain terminates in a stakeholder reason. Each step in the chain is a process+operand pair. Every architectural decision in Kubernetes can be interrogated against this chain, and any decision that cannot be is a decision that should be reconsidered.

Four readings, one system. The readings disagree about nothing; they illuminate different facets of the same object. This is what it looks like when a definition is doing work.

2.9 What This Buys Us

Before leaving this chapter, a preview of where this vocabulary will take us.

In Chapter ?? we will use this definition to sharpen the language of design—to distinguish components from modules from services, interfaces from boundaries, layering from dependency—because each of these is a way of talking about either entities, relationships, or the mapping from function to form.

In Chapter ?? the form-function-concept vocabulary becomes the backbone of conceptual design: we will formally separate *operand* (what the system acts on), *attribute* (what it changes about the operand), *function* (the process+operand pair), *concept* (the idea linking function to form), and *form* (the realized structure). Crawley’s mapping-from-function-to-form is the gear on which the entire chapter turns, and conceptual integrity (§2.5) is the discipline that keeps the gear from slipping as the system grows.

In Chapter ?? decomposition strategies—functional, domain-based, object-oriented, volatility-based—are evaluated against the criterion that *decomposition should respect the grammar of what flows between parts*. The Mars Climate Orbiter failure was a grammar mismatch at a decomposition boundary. So is the log-as-synchronous-mutation antipattern. So is most of the microservice pain cataloged in the last decade.

In Chapter ?? deep modules—the subject of Ousterhout’s program [ousterhout2018]—will be framed as abstractions whose *form* is maximally uninformative about their *internals* while their *function* is maximally informative about what they do. Deep modules are just a special case of a good function-to-form mapping.

In Chapter ?? Unix appears not as a historical curiosity but as the cleanest large-scale realization of the principles this chapter laid out: small pieces with single clean functions (Task 1 done well), sharp module boundaries (Task 2), uniform functional interactions through text streams (Task 3), and the emergence of unbounded compositional power from those disciplines (Task 4).

The definition is not decoration. It is the tool with which every subsequent chapter will cut.

2.10 Summary

- A system is an interacting collection of elements, bounded from its context, whose relationships sustain a function that no element realizes alone, and that function exists to answer a “why” that a stakeholder endorses.
- Four independent traditions—engineering (Crawley, NASA), cybernetics (Ashby), biology (Maturana and Varela), and philosophy of action (Anscombe)—converge on this structure, each contributing a different piece. Lakoff’s prototype theory explains why the convergence is evidence rather than coincidence.
- Systems have form (what they are), function (what they do, written as process+operand), and a concept (the idea linking form to function). When a single concept rules the whole system it has *conceptual integrity*, and integrity is what makes new features compose with old ones rather than fracture the mental model.

- Thinking about something as a system is a four-task loop: identify the system and its function; identify entities, boundary, and context; identify formal and functional relationships; predict emergence.
- Complexity is a property of the system; complicated is a property of our comprehension. The architect's job is to produce systems of the necessary complexity that are not complicated.
- The Mars Climate Orbiter failed at a relationship, not at an entity. The A320 at Warsaw failed at emergence under an unmodeled scenario. Kubernetes succeeds because its reconciliation loop realizes the autopoietic closure property over a variety-matched primitive set.

In the next chapter we turn from the question “what is a system?” to the question “what is the language with which we design one?”

3 Principles of Software Design

In this chapter, we introduce the fundamental principles of software design, focusing on the methodology and processes that guide the creation of effective software systems. This book is not about programming techniques or the use of specific programming languages. Instead, it emphasizes the conceptual frameworks and decision-making approaches necessary for designing robust, maintainable, and scalable software systems.

3.1 The Language of System Design

It is essential to distinguish between *design* and *programming*. While programming focuses on the implementation of specific algorithms or logic using a programming language, software design deals with how to organize and structure the system as a whole. Great systems are built by great architects, and to become a great software architect, one must develop the ability to step back from the immediate coding task and consider the larger picture.

To talk about system design, we need a robust language to eliminate ambiguity.

A **System** is a composition of interacting elements based on some set of rules, exhibiting some behavior defined by its purpose. A system has boundaries and is defined by its structure. E.g. a database-management system has a set of components (elements), interrelated data and a set of programs, e.g., SQL parser, storage manager, and a query optimizer. It has a boundary defined by its interface: an input language and a set of configuration knobs. Its purpose is to provide a convenient and efficient way of storing and retrieving database information. Here we are concerned with human-made (designed) systems, so we assume a system has a well-defined purpose or goal.

A **Subsystem** is a system that serves as a part of a larger system. An example of such would be memory subsystem in an SoC (System-on-Chip). As a system in its own right, it may have components like the scheduler, controller, and RAM. It has its own purpose — storing and providing access to data and instructions. And as a part of a larger system, it plays the role of an element, contributing to the overall goal of the SoC, performing some computation task.

3.2 System Decomposition

One of the primary challenges in software design is managing complexity. As systems grow in size and functionality, the need to decompose the system into smaller, manageable components becomes more critical. As Dijkstra noted in 1975, effective programming boils down to "very effective thinking" to manage complexity and separate concerns in a meaningful way.

The goal of system decomposition is to break down the system into functional components that can be more easily designed, understood, and maintained. This often involves identifying the various system components and their relationships and focusing on how these components interact.

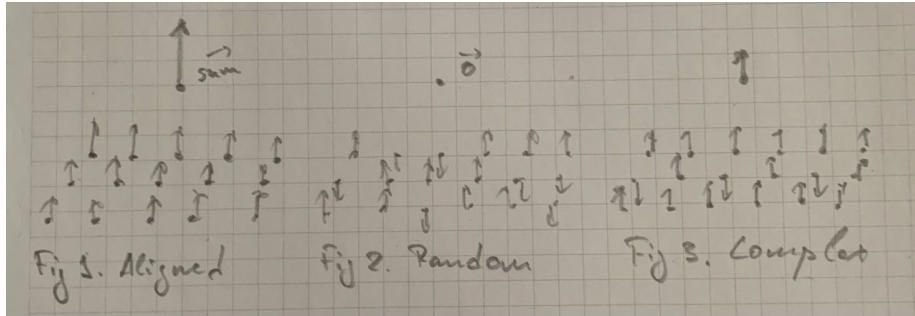


Figure 1: System complexity: motivational example

3.3 Complexity and Its Causes

In order to talk about complexity, it would be reasonable to give it a definition in the context of system design, so that one can reason about it. How do we understand whether a system is complex or not? We usually call a system complex if it is hard to make judgments about. While the complexity often correlates with the size of the system, it is not the size that makes a system complex. A large system can be easy enough to reason about if we can build a compact mental model of its parts and their relations to each other. Consider the following example of a spin glass system — a lattice with some particles with a spin at its nodes 1. Notice how the complexity of the system description (a high-level parameter, such as, e.g., overall magnetic field) depends on the entropy in the system. In both cases, when the entropy has its extrema, the model of a system is simple (leftmost & in the middle). The complexity creeps in somewhere in the middle, when there are non-trivial patterns exist that prevent one from giving a straightforward description, break the linearity of the system, introduce corner cases, etc.

This idea of complexity description is tightly coupled with the amount of information. The more patterns there are in the system, the more information is needed to describe it. In the context of system design description, we can reason about this in the following way. Every system, mechanical, biological, or conceptual, is built out of some "components". Let us call these components **parameters** λ_j . A system can then be described in terms of its parameters and relations between those. We parameters as nodes and relations as edges we can represent such a description in the form of a graph. The structure of such a graph would give us clues about how complex the system is, using the same entropy-based approach. Again, simple systems emerge at both ends of the spectrum. With the small number of relations, we have a simple graph of isolated components that are easy to reason about (think of a well-decomposed system with clearly defined, concise interfaces). Likewise, in a complete graph with the same relations between nodes and the same kinds of parameters, we get a parallel processing structure that is also easy to reason about (think of a set of equivalent services in a distributed system that have the same messaging

interface and are interchangeable). When the system moves from these two extremes to the middle, more patterns are introduced, thus, more types of parameters exist with more relations and more patterns in those relations. The closer the graph, describing the system, to some middle point on this scale, the more patterns it can capture — the more complex behavior a system can exhibit. If we are building a space ship navigation system, it has to include a model of all the relevant physical effects to reach the desired trajectory prediction accuracy. If we are building an operating system, it has to be able to map its internal representations to a wide variety of existing hardware.

This thought experiment leads to an important result in a natural way: complexity in software systems is inevitable. The more useful behaviors we want a system to produce, the more complex the graph becomes, meaning there is a certain degree of immanent (inherent) complexity to systems requiring certain behavior. Immanent complexity — the required (inherent) complexity for the system to perform its function. As the system evolves, this part of the complexity is likely to increase over time: new functionality is added, the integration surface increases, and new users with different usage scenarios emerge. From the complexity theory point of view, this can be described in the following rough terms. Assume the system performs a function $F(x)$ where x is some input to the system and F is a set (composition) of functions $f_1, f_2, \dots, f_n$ that work together to perform F . There is an optimal number of parameters λ_j that describe F . As we increase the number of functions (n), the number of parameters (λ) grows, so any possible implementation of the system requires more parts or interconnections.

The non-inherent complexity comes from the ways the system is implemented. Any engineering system built is a tradeoff made within a multidimensional parameter space (we increase the wheel diameter of a race car — the grip is now better, but acceleration suffers). The more restrictions each axis (parameter) has, the fewer options the system designer has. The fewer the options, the easier it is to simulate, compute, and reason about the tradeoffs. Software systems are mostly free from the vast number of restrictions and limitations that other engineering fields impose on a given project. There is no gravity, no fire safety requirements, no emission standards, etc. The lack of strict limits dramatically increases the search space for possible designs. It makes designing systems a mysterious, unstructured process. Once similar systems have been built multiple times, experience emerges. In a sense, an experienced architect uses the knowledge acquired in the past to impose additional limitations on the search space to limit the number of possible designs, thus trimming the unpromising designs.

Both inherent and non-inherent complexity can and should be managed. The immanent complexity is reduced by carefully choosing the set of functions f_i and how they are mapped to the internal parameters λ_j . The non-inherent complexity is managed through the system design processes, i.e., appropriate decomposition and composition, and all the well-known software engineering techniques. One prominent example of such a technique is the use of design patterns [Gam+95], which we would like to take a closer look at due to its

prevalence. While the patterns provide a powerful tool to communicate the intention and lay out ready-to-use recipes for certain common problems, designing a system using them as a primary tool leads to suboptimal decomposition, which we discuss extensively in chapter 5. We dive into the topics of mapping and decomposing in chapter 4.

Some of the primary causes of complexity include:

- **Dependencies:** A piece of code cannot be understood or modified in isolation. Dependencies between components can lead to tight coupling, making it harder to maintain and extend the system.
- **Obscurity:** Important information, such as unclear variable names or inadequate documentation, may be hidden or not easily understood.
- **Incremental Complexity:** Small dependencies and obscurities can accumulate over time, increasing the overall complexity of the system.

Understanding and managing these sources of complexity is crucial to effective system design.

3.4 Design and Architecture

Design is both a process and a result. Throughout the book, we will use the terms *design* and *architecture* interchangeably. Design decisions are driven by requirements—functional, non-functional, and quality attributes—that must be satisfied by the system.

Architecture, in the context of software systems, refers to the high-level structure that dictates how components are organized and how they interact. It is closely related to the system’s design but focuses on organizing the system to meet both functional and non-functional requirements. It can influence critical system qualities like performance, scalability, security, and maintainability.

3.5 Architecture and Design Drivers

Software architecture and design decisions are influenced by various “drivers”, which are the constraints or goals that guide the design process. These drivers can include technical constraints, budgetary limitations, performance requirements, or user needs.

The key goal of architecture is to tame complexity by selecting the right components and defining clear relationships between them. However, there is no single “right” architecture for all systems. The architecture must be tailored to meet the specific requirements and constraints of the project.

3.6 Software Architecture and Design

Architecture acts as the skeleton of a system. It provides a structural framework within which all the components of the system interact. However, architecture is orthogonal to functionality. For example, the same functional system could

be implemented in different architectural styles, such as a 2-tier, 3-tier, or peer-to-peer (P2P) architecture.

The architecture should provide a solution to the system's requirements while accommodating the necessary trade-offs. A well-designed system architecture not only addresses the functional requirements but also focuses on important non-functional attributes like scalability, performance, and security.

3.7 Requirements Definition

Every system is built in order to solve a real-life problem. The trick is to describe the problem in such a way that it can be understood by other human beings who would translate that understanding into a programming language.

While requirements are the starting point in the process of designing a system, they play multiple roles throughout the process of the system life cycle. High-level requirements provide a basis for agreement between the system designer and other stakeholders on what is to be built. They allow us to reason about and assess system's qualities (whether or not the system behaves the way it should). They guide the scheduling of the development work and provide the basis for any sort of (formal) verification.

For an architect, requirements are the primary tool to drive the system design and sequentially lower the risks. Requirements serve as a compass when navigating the open design-space waters, so setting the process of *requirements engineering* right is crucial. No matter your speed, sailing south instead of west will not get you closer to the destination. The **where to** is far more important than the **how**. Curiously, people suck at describing things precisely (e.g., exact instructions challenge). Natural language and our cognition just are not wired for the job. Computational systems, however, deal only with formally specified instructions. So, the vague natural language needs translation. Large complex systems (today, any meaningful system is large and complex) are built by multiple people. So, the "where to" needs to be conveyed to one another. **Understanding** becomes the cornerstone of the development process, and any error at this stage gets amplified downstream if not handled immediately.

The translation is enough of an issue on its own, but reality adds more complexity to the process. Requirements tend to change. New requirements emerge, some are not obvious, some are contradictory, incomplete, or plain wrong. Software engineering is quite different from other branches of engineering in its flexibility that allows for accommodating change to a relatively great extent. Such changes demand an iterative process for requirements engineering (fig. 2) and a set of qualities for the requirements specification.

The guidelines on requirement qualities and the engineering process are well established and available in the standards (e.g. ISO on requirements engineering). Such documents usually describe pitfalls, which non-the-less, are still common in the industry. Here we briefly outline the key concepts and how the processes of requirements engineering and elicitation fit into the big picture of software design.

The quest for software requirements (todo: requirements vs specification)

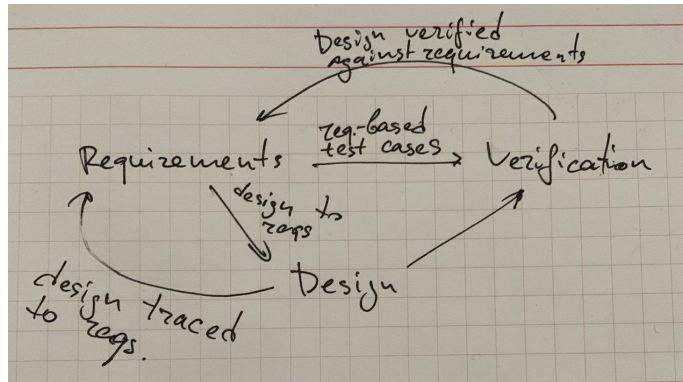


Figure 2: Requirements engineering process

Qualities of well written requirements:

- Traceability. You should be able to figure out why a certain requirement exists. What was the analysis that produced it, who is the beneficiary.
- Unambiguity. Use of unambiguous language, clearly defined terms. There should only be one possible interpretation.
- Verifiability.
- Completeness.
- Correctness.

The foundation of any design process is a solid understanding of the requirements of the system. Requirements describe the necessary functions and features that the system must provide and drive decisions on the solution space. They serve as input to the design process and are critical in guiding the system toward its goals. Requirements set constraints and goals in the **design** (things we control and decide on) and **objective space** (things our system achieves that are important to the customer). Clearly formulated requirements help navigate the design and objective spaces. There are strict requirements that set boundaries, usually described in a form of a "shall"-statement (e.g., "The house shall have 3 bedrooms"). These restrict the design space. The second class of requirements deal with the goals, and are usually expressed through "should"-statements (e.g., "The house should cost less than X dollars").

Requirements are typically divided into:

- **Functional Requirements:** What the system should do, including features and capabilities.
- **Quality Attributes:** Non-Functional requirements like performance, scalability, and reliability.

Title	Authors	Publisher / Year
<i>Software Requirements</i>	Karl Wieggers, Joy Beatty	Microsoft Press, 3rd ed. 2013
<i>Mastering the Requirements Process</i>	Suzanne & James Robertson	Addison-Wesley, 2012 (Volere)
<i>A Guide to the Business Analysis Body of Knowledge (BABOK)</i>	IIBA	IIBA, 3rd ed. 2015
<i>Requirements Engineering: From System Goals to UML Models to Software Specifications</i>	Axel van Lamsweerde	Wiley, 2009
<i>Requirements Engineering</i>	Ian Sommerville, Pete Sawyer	Wiley, 2nd ed. 1997
<i>Managing Software Requirements: A Use Case Approach</i>	Dean Leffingwell, Don Widrig	Addison-Wesley, 3rd ed. 2003
<i>Systems Analysis and Design</i>	Alan Dennis, Barbara Wixom, Roberta Roth	Wiley, latest ed.
<i>ISO/IEC/IEEE 29148:2018</i>	ISO/IEC/IEEE	International Standard

Table 3: Main books and references in software requirements engineering

- **Technical Requirements:** Constraints related to technology stack, deployment environments, data storage, and so on.
- **Operational Requirements:** Requirements related to monitoring, recovery, and backup processes.
- **Architectural Constrains:**

Clearly defining requirements ensures that the system will meet the user's needs and provides a basis for validating and verifying the system later in the development process. They establish a contract between the stakeholders and the builder on what product to build. Requirements also provide a basis for the estimate of project costs.

3.8 Examples of Poor Requirements

Many software failures can be traced back to poor requirements management. A few notorious examples include:

- **Mars Climate Orbiter (MCO):** A failure caused by units confusion between different systems (non-SI vs SI units) that led to the spacecraft's destruction.
- **FBI's Virtual Case File (VCF):** A project with unclear and changing requirements, leading to a 170 million loss without a usable system.

- **Denver International Airport Baggage Handling System:** Poorly defined and unrealistic requirements that led to the system's failure and delays in airport operations.

These examples illustrate the importance of clear, consistent, and well-defined requirements in successful system design.

3.9 Good Requirements Examples

On the other hand, well-defined and focused requirements can lead to successful systems. For example:

- **DC-3 Aircraft Design:** A simple, focused request for an aircraft with clear and achievable requirements such as range, speed, and passenger capacity.
- **Basecamp:** A project with a clear scope, incremental development, and a focus on simplicity. The requirements were user-driven, ensuring the product solved real problems without unnecessary complexity.

These examples highlight the importance of starting with clear, focused requirements that align with the overall goals of the system.

... requirements elicitation ...

SAFE

3.10 Conclusion

The key to designing effective software systems lies in understanding and managing complexity. By breaking down requirements into clear, actionable components and making informed design decisions based on these requirements, software architects can create systems that are both functional and scalable. Architecture provides the framework for the system, and effective design ensures that this framework meets both the technical and user needs. We will continue to explore how these principles can be applied to build systems that are not only technically sound but also meet the broader goals of the project.

4 From Requirements to Conceptual Design

4.1 Introduction to System Design

System design is an essential phase in software engineering, bridging the gap between clearly defined requirements and their eventual realization as software. Unlike straightforward processes, transitioning from requirements to design is iterative and nuanced. The boundary between requirements specification and design is fluid, with continuous interaction leading to refinement in both directions. The initial stages of system design are characterized by high ambiguity, demanding careful exploration to achieve clarity and precision.

The core phases of system design encompass stakeholder analysis, requirement definitions, and initial design exploration. Stakeholder analysis answers the critical question of "why" a project is undertaken, defining the value proposition clearly. Requirements definition identifies "what" must be achieved, while design focuses on "how" these requirements might be fulfilled.

In this chapter, we transition from the requirements gathering phase, which was covered in the first chapter, into the conceptual design phase of system architecture. This chapter explores how abstract design ideas and systems are formulated before diving into detailed software design. We also explore the critical steps involved in system design and architecture, from conceptualization to realization, and the process of transforming requirements into a robust and coherent architecture.

In this chapter, we delve into the essential concepts behind the process of system design. The focus is on the early stages of design, specifically on defining the requirements, identifying key concepts, and breaking down the system into manageable components. This chapter introduces the language of design and system architecture, which serves as the foundation for all subsequent design decisions.

4.2 From Requirements to Design

The journey from requirements to design is neither a strictly linear nor a well-defined process. It is an iterative process, where requirements evolve alongside the design itself. As design concepts take shape, new insights or needs may emerge that could alter the initial set of requirements. This iterative nature means that the line between requirements definition and design is often blurred, and modifications may need to be made at various stages of the process.

Every system, regardless of scale—whether a small tool or a large, complex system—requires an understanding of the fundamental questions:

- **Why are we building this system?** What value does it deliver, and to whom? This phase involves stakeholder analysis, determining who will benefit from the system and how.
- **What will the system do?** What are the functional requirements, and how will the system address them? This is the core focus of requirements

definition.

- **How could we do it?** This concerns the starting design and possible approaches to achieving the project's goals.

The clarity gained during this stage forms the basis for effective system design. It is also important to note that as we define requirements, they influence and shape the design, and conversely, the evolving design can influence or redefine the requirements themselves.

In many cases, there are multiple ways to achieve the desired results, which necessitates thoughtful exploration of the design options.

4.3 Phases in Systems Engineering

In the context of system design, there are clear phases that the design process follows:

- **Early Phase:** The ambiguity is high, and the goal is to reduce it. The focus is on gathering requirements.
- **Concept Phase:** The primary goal is to generate ideas, foster creativity, and develop abstract concepts for the system.
- **Design Phase:** As complexity increases, focus shifts to managing complexity through design, refining concepts into concrete systems.

This process gradually evolves from an abstract idea to a well-defined system. As complexity increases, it is important to maintain clarity and ensure that every step of the design process is aligned with the system's objectives.

System architecture is the embodiment of a concept. It involves allocating physical or informational functions to elements of form, and defining the structural interfaces among the objects. The architecture serves as a blueprint, defining how different components of the system interact and how the system will operate as a whole.

The architecture is composed of the following:

- **Function** – what the system does.
- **Form** – how the system is structured and how it physically or logically appears.
- **Concept** – the abstraction that links function and form together.

This concept-driven approach ensures that the architecture is not only functional but also adaptable, as it aligns the system's structure with its goals.

4.4 Conceptual Design

Once the requirements are understood, the next step is to transition into conceptual design. Conceptual design is an abstract process, where we begin to structure ideas and map out the basic elements of the system. At this stage, we do not have all the answers; rather, we are exploring the high-level structure of the system without delving into detailed technical specifics.

Originally derived from civil architecture, the concept provides a high-level abstraction or vision that guides the design process. This abstraction can be expressed informally as a mental image or formally as a detailed specification, dictating both the functionality and form of the designed system.

The conceptual design process involves clearly identifying the operand—the entity or subject upon which the system acts—and the attributes of this operand that deliver value. A clear example is a refrigerator, designed to slow the spoilage rate of food (operand) by maintaining a chilled environment (functional transformation). Multiple concepts, like chilling, drying, or irradiating food, can be explored to achieve the required functionality, ultimately selecting the most practical and efficient one.

In conceptual design, a *concept* is central. A concept is a fundamental idea or notion that serves as the anchor for the system or part of the system being designed. It acts as a bridge between functionality and form, mapping abstract ideas to potential solutions. For example, in software, the concept could be a module or subsystem that will deliver specific functionality.

Concepts in design are succinctly expressed through short descriptive phrases or simple visual icons, encapsulating the core functional mapping. Established concepts can often be referred to by common names directly associated with their form. Once a concept is clearly defined, the essential nature and list of parts become evident, simplifying the transition from abstract ideas to concrete implementations.

The key steps in conceptual design include:

- Identifying the core functions the system needs to perform.
- Mapping these functions to abstract forms or structures that will help realize them.
- Defining the relationships between these abstract elements and ensuring that they work together cohesively.

For instance, in the case of designing a refrigerator, the conceptual design could revolve around the idea of *keeping food fresh*. The function could be *slowing down spoilage*, and the form could be an insulated box that uses chilled air to achieve this.

4.5 Understanding Concepts and Their Mapping

The concept of a system or subsystem is more than just a set of features; it is a mental model that helps us understand how these features will work together. In

software, conceptual design often involves representing a system with high-level abstractions, such as modules, components, or service architectures.

A useful analogy for understanding this process is civil architecture. When designing a building, architects first conceptualize the idea of a space—such as a house or an office. They then map this concept to a form (e.g., a physical structure) that meets functional needs such as comfort, space utilization, and safety. Similarly, in software architecture, we begin with a conceptual model and progressively refine it into concrete, implementable components.

In the case of Git, for example, the concept revolves around the idea of *version control*. The function is to track changes to files over time, and the form is the data structure that represents this version history, such as commits and blobs (binary large objects).

Once the concept is defined, it is transformed into an architecture through a series of steps:

- **Establishing a solution vocabulary** – defining the language and terms that will guide the system’s development.
- **Mapping the concept to physical/informational form** – this is where abstract ideas are turned into tangible components.
- **Decomposing the system** – breaking down complex systems into smaller, manageable subsystems.

These steps lead to the creation of the system’s architecture, which can then be refined through functional decomposition to achieve the desired performance.

4.6 The Role of Functional Decomposition

Functional decomposition plays a key role in breaking down complex systems into manageable parts. In system design, especially at the conceptual level, functional decomposition helps identify core processes and map them to physical or abstract forms. However, it is important to avoid over-decomposition or overly granular functional breakdowns, as this can lead to unnecessary complexity and obscure the true goals of the system.

At this stage, it is crucial to balance the need for functional decomposition with the need for abstraction. The goal is to achieve a clear, logical organization of functions while preserving the integrity and simplicity of the system.

4.7 The Importance of Mapping Functions to Forms

The core of conceptual design lies in the mapping of functions to forms. A *function* represents what the system needs to do, while the *form* refers to how the function will be realized. This mapping process defines the system’s architecture and drives the design forward.

For example, in the case of a refrigerator, the function is to slow down spoilage, and the form is an insulated box using chilled air to preserve the food.

This form could take many shapes, such as a small cooler or a large industrial refrigerator, depending on the requirements.

- Operand: Food (the thing being transformed)
- Attribute to change: Spoilage rate
- Function: Slow spoilage
- Concept: A box that maintains a cold environment (a refrigerator)
- Form: Physical device using insulation and a cooling unit

Through this mapping, we transform abstract goals (keep food fresh) into tangible design elements (cooling unit, insulated box).

Let's analyze Git, the distributed version control system, using our conceptual framework:

- Operand: Files and directories
- Attribute: Content over time (version)
- Function: Track and retrieve historical versions
- Concept: A content-addressable, versioned file system
- Form: Commits, blobs, and trees connected via hashes

Git implements its core concept through a small set of abstractions (blobs, trees, commits), making it both elegant and powerful. The central idea is the dated file system—a snapshot of files and their states over time.

Mapping functions to forms is a creative process that requires understanding the problem space and the constraints within which the system must operate. In software, this often involves creating modular systems that can be easily extended and modified to meet future requirements.

4.8 Designing for Flexibility: Refined Concepts and Variables

Once a concept is established, it is refined through the design process. Concepts evolve as new requirements, constraints, or limitations emerge. A critical part of this refinement is the introduction of design parameters, which can include performance requirements, resource utilization, and cost.

In the context of software, these parameters might include memory usage, processing time, scalability, and user experience. Designers must constantly iterate and refine their concepts to ensure they align with the project's overall goals, adjusting the form and function as necessary.

4.9 The Role of Iteration in Conceptual Design

Conceptual design is an iterative process. As new ideas are tested and refined, the design evolves. In many cases, initial design choices will be revisited, adjusted, or discarded as more is learned about the problem domain.

Iterating over concepts helps avoid premature optimization or rigid thinking. In software design, it is essential to embrace flexibility and remain open to changes in requirements, especially when the design is not yet fully realized.

4.10 Conclusion: The Art of Conceptual Design

Conceptual design is a critical phase in system architecture that sets the foundation for everything that follows. By defining clear concepts and mapping functions to forms, system architects can create flexible, scalable systems that meet both functional and non-functional requirements.

In this chapter, we explored how to approach system design at a high level, breaking down the process of transforming abstract ideas into tangible systems. By understanding the importance of concepts, functional decomposition, and the iterative nature of design, software engineers and architects can craft solutions that are both effective and adaptable to future needs.

5 System Decomposition and Modularity

In this chapter, we delve deeper into the crucial phase of system decomposition. After discussing system requirements and conceptual design in previous chapters, we now focus on how to break down a system into smaller, more manageable components, each performing a specific role. This step, which involves the transformation of high-level design into subsystems and modules, is often the most controversial aspect of system design. We explore the importance of cohesion, well-defined interfaces, and the challenges involved in ensuring that subsystems integrate seamlessly to form a functional whole.

The origins of system decomposition can be traced back to the 1960s and 1970s with the rise of structured programming. In this era, top-down design methodologies promoted the idea of breaking down systems hierarchically, starting from high-level system goals and decomposing them into smaller subsystems and modules.

Key milestones in the evolution of decomposition strategies include:

- **Modular Design:** Introduced the idea of independent modules with well-defined interfaces that could be combined to form larger systems.
- **Dijkstra's Influence:** Edsger W. Dijkstra's work on structured programming and the "divide and conquer" strategy greatly contributed to the development of system decomposition techniques.
- **Object-Oriented Programming (OOP):** OOP refined decomposition by introducing objects that encapsulate both data and behavior, offering reusable components.
- **Data Flow Diagrams (DFDs):** These diagrams were used to visually represent data movement and functional components, aiding in the decomposition process.
- **Modern Approaches:** Today, modern techniques like microservices architecture further refine decomposition by dividing systems into small, independently deployable services for greater scalability and flexibility.

5.1 The Importance of Cohesion and Well-Defined Interfaces

At the heart of any successful system decomposition is the concept of *cohesion*. Cohesion refers to the degree to which the components within a module or subsystem work together to achieve a single, well-defined purpose. A cohesive system ensures that the parts fit together well, minimizing unnecessary complexity and redundancy.

For instance, when decomposing a system, one might imagine breaking down a car into various components, such as the wheels, engine, and seats. While each part has its own functionality, a lack of cohesion can lead to issues such as incompatibility or excessive complexity. For example, using skis instead of

wheels would create a less cohesive system, as skis are not suited for the primary task of moving a vehicle.

Along with cohesion, *well-defined interfaces* are crucial for ensuring that different subsystems can interact with each other. A subsystem should be self-contained, with a clear set of inputs and outputs. This modularity allows for easy updates and adjustments without disrupting the overall system. Ensuring that each module has a clear, understandable interface is key to creating a maintainable and extensible system.

5.2 The Evolution of System Decomposition Techniques

System decomposition has evolved significantly over the decades. In the 1960s, when computers became more widely available in large computer centers, the dominant approach to system decomposition was *top-down hierarchical design*. This approach focused on breaking down a problem into hierarchical levels, where smaller modules combined to create a fully functional system. This method was influenced by the top-down view of the world prevalent at the time.

The top-down decomposition approach worked well initially, but as systems grew more complex, it became clear that there were limitations. In the 1970s, the concept of *modular design* emerged, championed by Corden and Jordan's work on structured design. They advocated for decomposing systems into independent units with well-defined interfaces, inspired by principles from civil engineering. This idea paved the way for *structured programming*, a key step in the evolution of system design, which emphasized the use of functional decomposition.

As programming languages and computing power evolved, the 1980s saw the rise of the *object-oriented programming* paradigm, which shifted the focus from modular decomposition to thinking in terms of interacting objects. This approach, influenced by languages like Smalltalk, enabled systems to be designed as collections of objects that interact through well-defined messages.

In the 1990s, the rise of component-based architectures, such as CORBA, further emphasized modularity and distributed components. This laid the groundwork for the modern focus on microservices-based architectures, where systems are decomposed into small, independently deployable services.

5.3 Functional Decomposition and Its Limitations

One of the most natural ways to decompose a system is through *functional decomposition*, where the system is broken down into functional components that perform specific tasks. For example, if a system is required to produce reports, a separate *reporting component* would be created. If the system needs to sort data, a separate *sorting component* would be developed.

While functional decomposition is simple and intuitive, it has significant drawbacks. As the system grows and requirements change, the functional decomposition approach can lead to an explosion of components, each one fulfilling

a specific need. For example, in a house, functional decomposition might dictate that we build separate components for cooking, eating, bathing, and sleeping. However, if the requirements change—such as needing to add a new appliance like a grill—multiple components would need to be adjusted or rebuilt.

This leads to *component explosion*, where each change to a requirement necessitates changes across multiple components, increasing the complexity and reducing the maintainability of the system. Functional decomposition also introduces redundancy, as the same functionality may be duplicated across different modules.

The main limitations are:

- **Coupling between Modules:** Changes in one module may necessitate changes in others, leading to tight coupling between modules.
- **Explosion of Components:** Large systems often lead to an explosion of components, as the number of required functions increases.
- **Flattening of System Hierarchy:** As functions become increasingly complex, it can be difficult to manage them, leading to bloated orchestrators that handle many functions simultaneously.

The biggest challenge of functional decomposition is that it sometimes results in poorly organized systems, with modules becoming overly interdependent and hard to modify.

5.4 The Problem with Over-Decomposition

The key issue with functional decomposition is that it often leads to over-decomposition, where the system is broken down into smaller and smaller parts, each with its own set of responsibilities. This excessive granularity creates a web of dependencies, making it harder to maintain and evolve the system. In a large-scale system, such as an enterprise application, this approach can result in thousands of components, each serving a specific function, but requiring constant adjustments to accommodate new requirements.

Moreover, the overuse of functional decomposition can obscure the true goals of the system. When decomposing a house, for example, it might make sense to think about cooking as a separate component. But once we start adding multiple versions of cooking (e.g., microwave cooking, grilling, etc.), we risk losing sight of the underlying purpose, which is to provide a space for preparing food. This can lead to inefficiencies and unnecessary complexity.

5.5 Domain Decomposition as an Alternative

To overcome the limitations of functional decomposition, we turn to *domain decomposition*, where the system is divided based on its business domains or areas of functionality. For instance, instead of creating separate components for cooking, sleeping, and eating, we might create domains for the kitchen, bedroom, and living room, each with its own set of requirements and functionalities.

While domain decomposition provides a more holistic view of the system, it shares some of the same issues as functional decomposition, particularly when it comes to managing change. For example, if we build a kitchen and then later decide to add a living room, we may need to restructure the existing design to accommodate the new domain, leading to increased complexity.

- **Reimplementation of Functionality:** The same functionality may be reimplemented in different domains, leading to unnecessary duplication.
- **Difficulty in Extending Systems:** Adding new features to the system may require a complete restructuring of existing domains, especially if the system was not designed with flexibility in mind.

Despite these challenges, domain decomposition provides a better framework for organizing system components based on real-world contexts. It allows for a more natural mapping to the business requirements, ensuring that the system aligns with the needs of its users.

5.6 Object-Oriented Decomposition: A Modern Approach

Object-oriented decomposition is another widely used approach, where the system is modeled as a set of interacting objects, each encapsulating specific data and behaviors. This method focuses on creating classes that represent real-world entities and defines their interactions through methods (i.e., messages). Object-oriented decomposition emphasizes the principles of *encapsulation*, *inheritance*, and *polymorphism*, allowing for more flexible and reusable components.

Object-oriented decomposition typically follows these steps:

- Identify system requirements (functional and non-functional).
- Extract key objects and concepts relevant to the system's domain.
- Define class hierarchies using inheritance to establish relationships between objects.
- Assign responsibilities to each class, ensuring they follow the Single Responsibility Principle (SRP).
- Define interfaces for object interaction, ensuring encapsulation and data privacy.

However, object-oriented decomposition is not without its challenges. One of the main issues is that not all problems naturally lend themselves to object-oriented design. For instance, graph-based problems or procedural tasks like sorting may be difficult to model effectively using object-oriented principles. Additionally, excessive reliance on object-oriented decomposition can lead to complex inheritance hierarchies, making the system harder to maintain and scale.

- **Complexity in Large Systems:** As systems grow, managing class hierarchies can become challenging, leading to tightly coupled and complex designs.
- **Performance Overhead:** Object-oriented systems often introduce multiple layers of abstraction, which can slow performance and increase memory usage.
- **Difficulty Mapping Real-World Problems:** Not all problems are naturally suited to object-oriented approaches. For some cases, functional or procedural designs may be more appropriate.

In large systems, poorly designed inheritance hierarchies can result in systems that are difficult to extend and maintain.

5.7 Volatility-Based Decomposition: A Better Approach

One effective strategy for system decomposition is *volatility-based decomposition*, where we focus on identifying and isolating areas of the system that are subject to frequent change. By encapsulating volatile areas, we can minimize the impact of changes on the rest of the system. For example, in a system that tracks files, the file type may be a volatile area, as new formats can emerge over time. By encapsulating the file type logic in a separate component, we can ensure that changes to file formats do not affect other parts of the system.

Volatility-based decomposition emphasizes identifying the components of the system that are likely to experience frequent changes, such as data formats or user interfaces, and encapsulating these volatile areas in separate modules. This approach reduces the overall complexity of the system by isolating change, making it easier to adapt to new requirements over time.

Identifying areas of volatility requires careful analysis during the requirements phase, as volatility is not immediately apparent. Volatility can arise from changes in user needs, client expectations, operating environments, or content types, and must be carefully managed.

5.8 System Decomposition in Practice

In practice, decomposition strategies are not mutually exclusive. For instance, a complex system like Git combines different decomposition strategies to achieve scalability and maintainability:

- Functional decomposition helps define Git's core functionalities such as committing, branching, and merging.
- Object-oriented decomposition organizes Git's core elements (commits, trees, blobs) into objects with specific responsibilities.
- Volatility-based decomposition handles the system's different areas of volatility, such as user interactions, storage formats, and security concerns.

By combining these strategies, Git achieves a highly modular design that is flexible enough to handle changing requirements while maintaining performance and reliability.

5.9 Abstraction through Categorization

In this section, we will discuss some lessons that can be learned from abstract philosophy, cognitive theory, and languages, and how these notions and concepts can be applied to system decomposition.

5.9.1 Wittgenstein’s Language Games

Architects usually search for meanings or meaningful names in the systems to catch the decomposition beast by tail. Let’s examine the notion of the meaning as it is so elegantly laid down by Wittgenstein. In *Philosophical Investigations*, he argued that meaning is not a fixed essence hidden behind words but arises from their use in social practices – what he called “language games.” Ludwig Wittgenstein’s early work *Tractatus Logico-Philosophicus* treated language as a mirror of the world: sentences represented facts like pictures. Later, however, he came to see this as too rigid. In *Philosophical Investigations* (1953), he radically redefined meaning, not as correspondence to essences but as use in practice. Under the threat to be beaten by professional philosophers here are the core ideas that can be useful for our purposes.

First of all the notion of a language and language games - he argues that language is not one uniform system but a multitude of “games” with different rules. Asking for directions, writing a theorem, joking, or programming all use words differently. Meaning is tied to the rules of the game being played. Confusion arises when we import rules from one game into another.

Then he introduces the notion of a meaning as use. Words don’t gain meaning by pointing to hidden essences (“ostensive definition”). Instead, they mean what they are used for in a community. “Understanding” a word is knowing how to use it competently in practice.

Then he turns to the rules of putting the words together – grammar. Every language game has an implicit grammar: shared, rule-like practices that make communication possible. Grammar is rooted in forms of life—the human activities, needs, and instincts (e.g., care, survival, play) that ground meaning.

Many concepts (e.g., “game”) have no single essence. They are held together by overlapping similarities—a network of resemblances, not one defining trait. This explains why rigid definitions often fail in ordinary language.

Then he generalizes it to the highest cognitive level – philosophy in general. Traditional philosophy looked for essences of concepts – justice, mind, time. Wittgenstein saw this as a mistake: philosophical problems often come from “language going on holiday” — using words outside their natural grammar. The philosopher’s task is not to define but to clarify: to map the uses of words and dissolve confusion.

So why it matters to us? No single definition captures complex concepts. Context and practice matter. Rules are local. Each domain or community has its own grammar. Understanding is participatory. To “get” a concept is to play the game. Philosophy shifts to mapping practices rather than theorizing essences.

To grasp the idea nothing is better than a good example. Here’s how Stephen West is describing the language game in this good example in one of his podcast episodes on Wittgenstein ??:

So there’s a tactic that’s become pretty popular in what some people would call the debate space of the Internet these days. There’s a trick someone will do where at the very beginning of the conversation, they’ll ask the other person to define the exact thing that they’re going to be talking about that day. Well, sound kind of like this.

- Just to start out today, can you please give me your definition of God? Can you give me a definition of abortion or insurrection or justice, whatever it is that day? And then the other person will usually take the bait. They try to give their take on it. Maybe they’ll say an insurrection is when a group of people try to overthrow some form of authority out there. Then the other person will say, back to that.

- Well, based on your definition, is a prison riot an insurrection? Then that’s people overthrowing an authority. If a union fires a manager that’s harassing employees, is that an insurrection? How about if my two kids both kick me in the shins at the exact same time? Is that an insurrection? I mean, if you can’t even define what it is we’re supposed to be talking about today, are you even qualified to be here? All the while, this person’s usually winning points with the crowd that’s watching the debate. I mean, if the other side can’t get to the essence of what we’re talking about, then what are we even talking about?

The key insight is that language doesn’t rest on one universal structure but on a family of practices tied together by family resemblances rather than strict definitions. Words gain their meaning from context and from the shared form of life of their users. So “understanding” a word means being able to participate competently in its game, not mapping it to some Platonic ideal.

So here are the subjective applications of those lessons for our purposes.

- Abstractions don’t come from essences, but from use. Wittgenstein critiques the Augustinian picture of language where words are “names” for essences. Instead, meaning arises in practice—through the rules of a community and the uses of words in context.

Don’t design abstractions as if they have a hidden, universal “essence.” Instead, design them from use cases, interactions, and community expectations. For example, in a distributed runtime, the “Job” abstraction

doesn't exist because there is some metaphysical "Job-ness." It emerges because developers and schedulers use certain patterns of task grouping, logging, and resource negotiation. Abstractions should be descriptive, not prescriptive. They codify the patterns developers already rely on, rather than impose alien conceptual frames.

- Grammar = Rules of Play. Wittgenstein stresses that words require a grammar: implicit, community-generated rules of use. Different language games (geometry vs. daily speech) operate under different grammars, and confusion arises when one grammar is applied to another.

Each subsystem has its own grammar (rules and invariants). The scheduler has different rules than the compiler IR; the storage system has different rules than the networking stack. Architect should perform the decomposition by respecting each subsystem's grammar. Don't force the rules of one subsystem onto another (e.g., imposing strict consistency models from databases onto real-time streaming). Misaligned decomposition often comes from mixing grammars—like treating "logging" as if it were synchronous state mutation, when its grammar is actually "asynchronous observation."

- Family resemblances, not single definitions Wittgenstein's "game" example shows that many concepts can't be defined by one essence but by a network of overlapping similarities.

When you define an abstraction like "Service" or "Orchestrator," don't hunt for a single property that unites all instances. Instead, look for family resemblances: some services are stateful, some stateless; some are user-facing, some are infrastructure; but they share enough overlapping traits that the abstraction is useful. This means decomposition strategies should tolerate blurred edges: a component may play multiple roles depending on context (a worker might sometimes act as a cache).

- Ordinary vs. Formal Language Games = Domain vs. Infrastructure Design. Wittgenstein contrasts the rigid rules of Euclidean geometry with the fuzzy edges of everyday speech. Both are valid, but confusion arises when you import one grammar into the other.

In infrastructure layers (compilers, type systems, schedulers), you can enforce rigid, formal grammars. In domain layers (business logic, data modeling, user interaction), fuzzier, evolving categories are natural and should be supported. Good architecture layers the formal beneath the informal: rigid grammars provide stability, while higher-level fuzzy abstractions remain flexible and contextual.

- System architect as cartographer, not legislator. Wittgenstein redefines philosophy as "therapy": clarifying practices, describing language games, and laying out family resemblances. Not inventing hidden essences.

This means that the architect’s role is not to decree perfect abstractions from on high but to map practices, reveal grammars, and provide clear interfaces. A good decomposition is more like a language atlas than a taxonomy: it shows how modules are used, how they interrelate, and where rules differ.

This reduces “talking past each other” in large engineering teams, where disagreements often stem not from code but from mismatched grammars of understanding.

For a software architect, this shifts how we think about abstractions: they aren’t “true” in the sense of eternal definitions, but useful conventions within a context of practice. Much like language games, software modules make sense in terms of the interactions, constraints, and behaviors in which they are embedded.

Contextual meaning: An abstraction’s meaning depends on how it’s used in the larger system, not just its formal definition.

Family resemblance: Different modules (e.g., logging frameworks, schedulers, memory managers) may share enough similarities to be treated as part of the same “game” even without identical properties.

Pragmatic rules: What matters is that participants (developers, users, other subsystems) know the “rules of play”—the interfaces and conventions—not that the abstraction conforms to a universal taxonomy.

— Almost proved that volatility-based decomposition is the right one :)

5.9.2 Cognitive categorization

Now let’s raise the abstraction level even higher for our decomposition strategies and talk about more generic sets of meanings — categories.

In George Lakoff’s?? framework, a category is a way the human mind groups experiences, objects, or concepts together so they can be understood, reasoned about, and acted upon as a unit. Unlike the classical (Aristotelian) definition — where categories are sets defined by necessary and sufficient conditions, Lakoff’s theory in “Women, Fire, and Dangerous Things”?? challenges the classical view that categories are defined by clear-cut rules and necessary features. Instead, he argues that human categorization is often prototype-based: categories are organized around central, typical examples (a robin is a more typical “bird” than a penguin). Categories are also graded and fuzzy, allowing for partial membership and overlaps rather than rigid boundaries. He shows that “radial categories” emerge, where a central concept expands into related subtypes through metaphorical or functional links. This perspective highlights how human cognition is not strictly logical or taxonomic but shaped by embodied experience and cultural frames.

Lakoff also emphasizes the role of conceptual metaphors—structures we use to understand abstract domains in terms of more concrete, embodied ones (e.g., “time is money” or “argument is war”). These metaphors shape not only how we think but also how categories are formed and extended. His broader claim is

that human cognition is embodied and context-dependent: categories reflect our bodily experiences, social practices, and histories. Thus, categories are dynamic, evolving, and contextually sensitive, not static or universal. This view reshapes fields from linguistics to philosophy, showing that meaning and thought are grounded in lived experience rather than abstract logical definitions.

As a system architect, you can treat those insights as directly relevant to system decomposition, how you break a system into parts, and abstraction creation how you name, layer, and reason about those parts.

Though, due to length limitations we can't present full theory here and leave that to the readers, here are some lessons a software architect can take from Lakoff's observations.

- Categories and abstractions are not just hierarchies. Human categories aren't crisp Aristotelian sets; they're often fuzzy, graded, and prototype-based ("a robin is more of a bird than a penguin"). Which means that you shouldn't expect your decomposition to always fall into neat hierarchies. Real systems have modules that overlap, hybrid responsibilities, and 'fuzzy boundaries'. Example: A GPU compiler pipeline might treat 'memory management' as part of both the runtime and the optimizer. Forcing a rigid tree often creates distortions. Instead, acknowledge graded membership and design for cross-cutting concerns explicitly (logging, resilience, security).
- Prototypes and exemplars matter. Categories often center around prototypes (typical examples) rather than strict rules. As an architect, when creating abstractions, emphasize a prototypical use case rather than an abstract generalization. Example: Instead of designing a generic "DataBuffer" upfront, start with the prototype "Tensor on GPU memory." That prototype anchors the abstraction, and later variants (tensor on shared memory, tensor on disk) can radiate from it.
- Radial categories and core + extensions. Abstractions as categories can be radial, with a central core and multiple extensions that are linked metaphorically or functionally. Good abstractions often have a core API with radial extensions. Example: In a distributed runtime, the "Process" abstraction may radiate into "WorkerProcess," "ServiceProcess," and "OrchestratorProcess." They're not arbitrary; they extend the same conceptual center. This helps avoid monolithic design while still grounding variants in a stable conceptual nucleus.
- Metaphors shape architecture. As humans we think metaphorically ("time is money," "software is a machine"). These metaphors structure reasoning. It's crucial to be conscious of the metaphors guiding your decomposition. If you think of "pipeline" as water flow, you'll emphasize linearity and bottlenecks. If you think of it as "graph," you'll emphasize parallelism and flexible composition. Choosing metaphors deliberately can unlock different decomposition strategies.

- Family resemblance vs. rigid taxonomy. Some categories are held together not by one defining property but by a family resemblance. This can mean that in a system, modules don't always share one trait. Sometimes they're grouped because of overlapping features or patterns of use. Example: In kernel DSLs, CUDA, Triton, and Mojo may all be grouped as "GPU-centric kernel languages," even though their type systems and lowering differ. The resemblance, not strict identity, justifies grouping.
- Embodied cognition and experience "fit". Lakoff's point here is that our categories are grounded in human embodiment and experience. So, abstractions should be cognitively ergonomic. The "shape" of the abstraction should feel natural to the developers and users. Example: In PyTorch, "Tensor" feels intuitive because it matches (as an n-d array) both the mathematical prototype and the programming metaphor. If an abstraction forces users to think in alien terms, it will resist adoption.
- Category membership shifts depending on context. System decomposition should embrace contextual polymorphism. Example: The same "Tensor" object might be considered part of the data model in one layer, and part of the communication protocol in another. Abstractions should allow re-framing depending on architectural view (logical vs. deployment).
- Categories evolve with use, culture, and history, so don't over-fix decomposition at design time. Allow abstractions to evolve as the system matures and as new "prototypes" emerge. Example: a system might start with CPU/GPU as device categories, then later extend to TPU/NPU/RISC-V without redesigning the entire abstraction.

Need some finale here

5.10 Conclusion: Choosing the Right Decomposition Strategy

The key to successful system decomposition lies in choosing the right approach for the specific context. While functional decomposition and domain decomposition are natural starting points, they can lead to issues with complexity, redundancy, and over-decomposition. Object-oriented decomposition provides more flexibility, but it can be challenging to apply in certain domains.

Strategy	Pros	Cons	Best Use Case
Functional	Mirrors requirements	Poor change management, duplication	Requirement discovery
Domain-Driven	Business-aligned language	Redundant logic	Business process modeling
Object-Oriented	Encapsulation, abstraction	Deep hierarchies, poor fit for some domains	Low-level component modeling
Volatility-Based	Localizes change, scalable design	Requires expertise	Long-term, evolving systems

Volatility-based decomposition offers a promising alternative, as it focuses on isolating areas of change and minimizing their impact on the rest of the system. By carefully considering the volatility of different components and structuring the system around these volatile areas, we can create a more maintainable and adaptable design.

Ultimately, the goal of system decomposition is not to apply a single technique uniformly across the entire system, but to find a balance between different decomposition strategies that best align with the system's requirements and constraints.

6 Abstraction and Module Design in System Architecture

The design of complex systems is often dependent on the ability to properly abstract and modularize components. Abstraction allows us to focus on essential features while hiding unnecessary details, simplifying system design and making it more manageable. In this chapter, we will explore the concept of abstraction, its application in system architecture, and techniques for designing deep, efficient modules. We will look at examples from both theoretical foundations and real-world programming scenarios, ultimately building an intuition for creating modular systems that are both flexible and maintainable.

6.1 Understanding Abstraction in System Design

Abstraction is the process of simplifying complex systems by focusing on relevant characteristics while hiding the complexity that is not pertinent to the current problem. It helps reduce the cognitive load required to interact with complex systems by allowing developers to only consider the most relevant aspects of a system.

In the realm of physics and mathematics, abstraction is used to strip away details that do not influence the problem at hand. For instance, in physics, when analyzing the Earth's gravitational field, we might abstract the Earth into a point mass, ignoring its detailed shape and structure to focus purely on gravitational interactions. Similarly, in programming, abstraction involves removing unnecessary implementation details and exposing only the necessary interfaces to the user.

A common real-world example of abstraction in software is a game system. For instance, in a space shooter game, we might have different types of objects such as the player's ship, enemies, and obstacles. Each object has similar characteristics (e.g., position, direction, weapon) but varies in behavior. Initially, these objects might be modeled in separate classes, leading to redundant code. By abstracting common characteristics into a base class, we reduce code duplication and streamline the system.

6.2 The Concept of Deep Modules

A key concept in system design is the notion of deep modules. Deep modules are abstractions that encapsulate complexity while maintaining a simple interface. The goal of deep modules is to provide powerful functionality without exposing users to the intricacies of the underlying implementation.

Consider the example of a memory allocation system in a low-level language like C. The system hides all the complexities of memory management—such as allocating and freeing memory blocks, managing fragmentation, and optimizing for performance—behind a simple interface like `malloc`. For the programmer using this function, memory allocation appears to be a straightforward task,

but in reality, the underlying system is performing complex operations to ensure efficient memory use.

The essence of a deep module is that it hides unnecessary complexity from the user while still providing rich functionality. A deep module minimizes the amount of interaction required by the user with the underlying implementation, allowing them to focus solely on solving the problem at hand.

6.3 Avoiding Shallow Abstractions

One of the key challenges when designing abstractions is avoiding shallow modules—modules that expose too many implementation details or require users to be overly familiar with their inner workings. Shallow abstractions lead to overly complex interfaces and can result in tightly coupled systems, where changes in one module require widespread modifications across the codebase.

For example, if a game-saving system was designed such that the user needs to specify whether the data should be saved in XML or JSON format, this could lead to a highly coupled system where any changes to the data format would require changes throughout the code. A better approach would be to abstract the format-saving functionality into separate classes and provide a simple interface for saving data, without the user needing to know the format details.

Abstraction should always hide implementation details that are not relevant to the user. If a module's interface reveals too much internal information, it defeats the purpose of abstraction and complicates the system unnecessarily.

6.4 Designing Modularity: Generalization vs. Specialization

A critical part of system design is deciding whether to generalize or specialize the modules in a system. Generalization leads to greater reusability and flexibility, but excessive generalization can result in overly complex and unwieldy systems. Conversely, specialization tends to reduce complexity by focusing on specific use cases, but it can limit the system's flexibility.

For example, in object-oriented design, you might start with a general class, such as a 'GameObject' class, which encompasses common behaviors for all game objects. Over time, you might need to specialize this class further, for example, into 'PlayerShip', 'EnemyShip', and 'Obstacle'. While this specialization provides more precise control, it can also result in code duplication and reduced flexibility.

The key challenge lies in finding the right balance between generalization and specialization. The more general a module is, the more likely it is to be reusable in different contexts. However, this generality comes at the cost of complexity, as the module will need to handle a wider range of use cases. On the other hand, specialized modules are easier to understand and use, but they are limited in scope and may need to be re-implemented if the system evolves.

6.5 Managing Complexity Through Modularity

Modularity is essential for managing complexity in large systems. By breaking a system into smaller, independent modules, developers can isolate changes, reduce interdependencies, and make the system easier to understand and maintain. However, the process of modularization introduces its own challenges.

When designing a modular system, it is important to ensure that the modules remain loosely coupled. Tightly coupled modules can create situations where changes in one part of the system ripple throughout the entire codebase. To avoid this, developers should strive to define clear interfaces for each module and minimize shared knowledge between them.

An example of effective modularization is seen in modern compilers, which allow users to define optimization passes as separate modules. Each pass performs a specific transformation on the intermediate representation of the code, and the user can chain them together to customize the compilation process. This modular approach allows users to select and combine different passes to achieve the desired optimization, without modifying the underlying compiler code.

6.6 The Role of Layers in Deep Modules

Deep modules are often structured in layers, where each layer provides a different level of abstraction. Each layer handles different aspects of the system's functionality, and complexity is pushed downwards to lower layers where it can be managed more efficiently.

In the case of a memory management system, for example, the top layer might provide an interface for simple memory allocation and deallocation, while lower layers might handle complex tasks such as managing memory fragmentation, optimizing memory allocation across processors, and interacting with the underlying hardware. By pushing complexity downwards into lower layers, the higher-level layers remain simple and easy to use.

This layered approach is also evident in network protocols like TCP, which abstract away the complexities of packet transmission, routing, and error handling, exposing a simple interface for applications to send and receive data. The underlying layers handle the intricate details, such as connection establishment and data flow control, without requiring users to understand the complexities of the protocol.

6.7 Best Practices for Module Design

When designing modules, it is crucial to focus on both the internal complexity and the user-facing interface. A module should have a clear purpose, and its interface should be simple yet flexible enough to accommodate future changes without sacrificing ease of use. It is also essential to avoid overgeneralizing, as this can lead to unnecessary complexity and confusion for users.

To achieve these goals, consider the following best practices:

- Ensure that modules are focused on a single responsibility, adhering to the single responsibility principle.
- Hide unnecessary details from the user by providing a clean and concise interface.
- Maintain a balance between generalization and specialization, generalizing when flexibility is needed, and specializing when performance or simplicity is prioritized.
- Structure modules in layers to separate concerns and manage complexity.
- Design modules to be loosely coupled, minimizing dependencies and interactions with other modules.

By following these principles, software architects can create deep modules that are both efficient and easy to use, contributing to the overall quality and maintainability of the system.

6.8 Conclusion

The design of abstractions and modules is fundamental to building scalable, maintainable systems. By focusing on deep modules that hide unnecessary complexity while providing powerful functionality, developers can create systems that are both flexible and easy to interact with. Striking the right balance between generalization and specialization, as well as ensuring that modules are designed with clarity and ease of use in mind, are key aspects of successful system design.

As you continue to explore the art of system architecture, remember that creating deep modules requires both experience and discipline. With practice, you will develop an intuition for knowing when to generalize, when to specialize, and how to structure modules in a way that minimizes complexity and maximizes efficiency.

7 Practical Applications of System Design: A Case Study of Unix

In the first four chapters, we explored the foundational concepts of system decomposition and composition. We discussed how to break down complex systems into manageable parts, how to create modular and extensible systems, and how to design systems with clear interfaces. Now, in this lecture, we shift our focus to how these concepts are applied in real-world systems, with a specific focus on Unix. Unix, as one of the most influential operating systems in the history of computing, serves as an exemplary case study for understanding how design principles are implemented in practice.

7.1 Introduction to Unix and its Origins

Unix is an operating system that has had a profound impact on the development of modern computing. Initially developed at AT&T Bell Labs in 1969 by Dennis Ritchie and Ken Thompson, Unix was designed to be a simple, modular, and extensible system. It was born out of a desire to overcome the limitations of a previous operating system project, Multics, which failed due to its complexity.

Unix was initially written for the PDP-7 machine in assembly language, and by 1972, a functional version was released. In 1975, Unix was rewritten in the C programming language, which was developed by Dennis Ritchie as a successor to the B programming language. This was a groundbreaking development, as it allowed Unix to be ported to different hardware platforms more easily.

The Unix philosophy, which emphasizes simplicity, modularity, and the creation of small programs that do one thing well, has influenced countless other operating systems, including modern versions of Linux and macOS. Although Linux is not a direct descendant of Unix (since it was independently developed by Linus Torvalds), it shares many of its design principles and is often referred to as a "Unix-like" system.

7.2 The Unix Design Philosophy

Unix's design philosophy is grounded in pragmatic decisions made by its creators, and it is centered around several key principles. The philosophy is not a formal method but rather a set of guidelines that shape how problems are approached and solved. Below are some of the most important principles that define Unix's approach to system design:

- **Make each program do one thing well:** Each program should focus on a specific task and perform it efficiently. Programs should not try to do too much, and they should avoid unnecessary complexity.
- **Expect the output of every program to become the input of another:** Unix programs are designed to work together. Programs communicate by passing data through standard input and output streams, making it easy to combine them into complex workflows.

- **Don't clutter output with extra information:** Programs should provide only the necessary output, avoiding excess information that could clutter the user's view.
- **Design and build software to be tried early:** Software should be developed incrementally, with working prototypes created quickly. If a feature is clumsy or unnecessary, it should be discarded.
- **Use tools to solve problems:** Unix encourages the creation of simple tools that can be combined to solve complex problems, rather than relying on large monolithic programs.

These principles are part of a broader Unix tradition that favors small, simple, and modular designs that can be easily extended, reused, and maintained. They emphasize the importance of clarity and simplicity over cleverness and over-complication.

7.3 Unix as a Modular and Composable System

The key to Unix's success lies in its ability to decompose complex tasks into smaller, manageable components, or modules. The system is organized into layers, with the kernel at the core, surrounded by various system services, shells, and application programs.

The Unix kernel is responsible for managing hardware resources, scheduling processes, and providing basic system services. The kernel interacts with the system through a simple, consistent interface known as the system call interface. This interface allows higher-level programs to interact with the hardware without needing to deal with low-level details.

Around the kernel, Unix provides a variety of services and utilities, including compilers, linkers, and shells. The shell is the primary interface through which users interact with the system, allowing them to run programs and manage files. It provides a simple command-line interface that communicates with the kernel to perform tasks such as file manipulation, process management, and network communication.

Unix also treats everything as a file, including hardware devices and system processes. This abstraction allows for a unified interface, where data can be read from and written to files in the same way, regardless of whether the file represents a physical device, a process, or a regular file. This consistent treatment of files simplifies the design and composition of the system.

7.4 Composition and Modularity in Unix: Filters and Pipelines

One of the most powerful features of Unix is its ability to compose programs into more complex systems through the use of pipes. A pipe allows the output of one program to be used as the input to another, enabling the creation of pipelines where multiple small programs work together to perform a larger task.

The Unix approach encourages writing small programs, or “filters,” that each do one thing well. These filters can be combined in a pipeline to solve complex problems. For example, the ‘cat’ command reads the contents of a file, the ‘sort’ command sorts lines of text, and the ‘uniq’ command removes duplicate lines. By combining these simple programs using pipes, users can easily perform tasks such as sorting a file and removing duplicates with a single command:

```
cat file.txt | sort | uniq
```

This pipeline reads the contents of ‘file.txt’, sorts the lines, and removes any duplicates, all using small, single-purpose programs that work together seamlessly. This modularity and composability are central to the Unix philosophy, as they allow for great flexibility and efficiency in system design.

7.5 The Importance of Abstraction and Information Hiding

Abstraction is a key concept in Unix design, and it is implemented through the use of deep modules that hide unnecessary details from the user. In Unix, processes and files are abstracted through simple, consistent interfaces that allow users to interact with the system without needing to understand its internal workings.

For example, the file system in Unix abstracts away the complexities of disk management, allowing users to interact with files through a simple interface of file names and paths. The file system provides an abstraction layer over the physical storage medium, whether it is a spinning disk, an SSD, or even memory. This abstraction allows users to focus on higher-level tasks, such as organizing files and directories, rather than worrying about how data is physically stored.

Similarly, Unix abstracts the complexities of process management through the ‘fork’ system call, which creates new processes by duplicating the current process. The process model in Unix is simple and elegant, allowing for the creation of new processes from existing ones and enabling the creation of complex task hierarchies.

7.6 Lessons Learned from Unix Design

The Unix design philosophy offers several valuable lessons for modern software architects and engineers:

- **Keep it simple:** Unix emphasizes the importance of simplicity in design. Programs should do one thing well, and complex systems should be composed of small, simple modules that work together.
- **Use abstraction to manage complexity:** Abstraction allows you to hide unnecessary details and focus on the essential aspects of a problem. Unix achieves this by treating everything as a file and providing a consistent interface for interacting with system resources.

- **Focus on modularity and composition:** Unix's success lies in its ability to compose simple programs into more complex systems. This approach encourages the reuse of code and makes it easier to build and maintain large systems.
- **Design for flexibility and extensibility:** Unix was designed with the goal of being easily repurposed by developers other than its creators. Its modularity and clean interfaces make it easy to extend and adapt to new use cases.

7.6.1 McIlroy's Principles

1. Do one thing well.
2. Use plain text as a universal interface.
3. Compose programs via input/output.
4. Build software early and iterate quickly.
5. Favor tool-building over repetitive tasks.

7.6.2 Rob Pike's Rules

1. Bottlenecks occur in unexpected places.
2. Don't optimize until you measure.
3. Fancy algorithms are slow when n is small.
4. Simple data structures often suffice.
5. Data dominates: the right structure leads to obvious logic.

7.6.3 Raymond's Rules (Selected)

- Modularity: Connect small, clean modules.
- Clarity: Prefer clarity to cleverness.
- Composition: Programs should interoperate cleanly.
- Separation: Keep policy separate from mechanism.
- Transparency: Behavior should be inspectable.
- Economy: Optimize for programmer effort.
- Stream-Oriented Design: Use text streams for interoperability.

7.7 Conclusion

Unix stands as a prime example of how system design concepts, such as modularity, abstraction, and composition, can be applied to create a powerful and flexible operating system. Its design philosophy continues to influence modern operating systems, and its emphasis on simplicity, clarity, and efficiency serves as a valuable model for system architects today.

By understanding the design principles that underpin Unix, software engineers can gain valuable insights into how to tackle complexity and build maintainable, extensible systems. The Unix approach teaches us that the key to success lies in creating deep, focused modules that work together to solve larger problems while keeping the interface simple and clear.

8 Designing Build Systems and Automation Tools

In this chapter, we explore the process of designing build systems, an essential component in modern software development. Build systems are used to automate the transformation of source code into executable programs or deployable packages. These tools are critical when dealing with projects that involve multiple source files and complex dependencies. We will discuss the basics of build tools, their components, and the importance of automating tasks like compilation, linking, packaging, and testing.

8.1 What is a Build System?

A build system is a tool used to automate the process of transforming source code into a deployable executable. It involves compiling source files, linking them together, and resolving dependencies to generate a final program. In larger projects, build systems also automate tasks such as testing, packaging, and deployment.

At the core of most build systems is the concept of a *dependency graph*. In this graph, each file or module in a project is represented as a node, and edges indicate dependencies between files (e.g., one file depends on another). The build system must ensure that all dependencies are properly resolved and that files are compiled in the correct order. If a file is changed, the build system must ensure that only the necessary parts of the project are recompiled, saving time and computational resources.

8.2 Why Do We Need Build Tools?

As software projects grow in complexity, the need for automation increases. Without build tools, developers would need to manually compile and link hundreds or even thousands of source files, which is time-consuming and error-prone. Build tools help automate this process and ensure that all dependencies are correctly managed.

For example, in languages like Java, where code is often split across multiple files (e.g., classes that are defined in one file but used in others), build tools help automate the process of converting source code into an executable program. This process involves using a compiler to turn each source file into an object file and a linker to combine these object files into a single executable.

8.3 Common Build Tools and Package Managers

Build tools are available for many different programming languages. For example:

- In Java, popular build tools include **Maven** and **Gradle**.
- In Python, **setuptools** and **pip** are used for building and installing packages.

- In C++, there are tools like **Make** and **CMake**.
- In Go, the language comes with a built-in build tool as part of its standard toolchain.
- In Rust, the build tool is called **Cargo**.

These tools help automate the build process and also manage dependencies (e.g., third-party libraries or packages that a project needs). Some of them, like **Maven** and **Gradle**, also include features for managing project dependencies, running tests, and generating documentation.

In C++, the **Make** tool is one of the oldest and most widely used build systems. It uses a file called a **Makefile** to define the dependencies between source files and the rules for how to compile them.

8.4 The History of Make: The First Build Tool

The first known build tool is **make**, which was created by Stuart Feldman at Bell Labs in the early days of Unix programming. **Make** is based on a simple concept: it defines a set of rules and dependencies that describe how to transform a project's source files into an executable program. Each rule specifies a target (e.g., a file to be generated), the dependencies (e.g., source files), and a recipe (e.g., a command to be run) to produce the target.

The power of **make** comes from its ability to automatically track changes in files and only rebuild the files that have changed or are dependent on changed files. This selective recompilation helps save time in large projects where recompiling everything after each change would be impractical.

8.5 Makefile Basics

A **Makefile** is a text file that defines a set of rules for building and managing dependencies. Each rule consists of:

- A **target**, which is typically a file that the rule will generate.
- A list of **dependencies**, which are the files needed to create the target.
- A **recipe**, which is a command that tells the build system how to create the target from its dependencies.

Here is a simple example of a **Makefile**:

```
target: dependency1 dependency2
    command_to_build_target
```

In this example, the target depends on **dependency1** and **dependency2**, and the specified command will be executed to generate the target.

A typical **Makefile** might look like this for a small project:

```

all: main.o utils.o
    gcc -o program main.o utils.o

main.o: main.c main.h
    gcc -c main.c

utils.o: utils.c utils.h
    gcc -c utils.c

```

This **Makefile** specifies that to build the **program**, **main.o** and **utils.o** are required. To generate each object file, the corresponding source file must be compiled.

8.6 Challenges in Using Make for Large Projects

While **make** is powerful, it becomes more complicated as projects grow. With larger projects, the **Makefile** can become increasingly difficult to manage. For example:

- Managing complex dependencies can make the **Makefile** difficult to read and maintain.
- Repetitive tasks, such as specifying the compiler flags, can lead to redundant code.
- The **Makefile** format is simple, but it lacks the flexibility of modern build systems, making it hard to support more complex use cases.

These limitations have led to the development of more advanced build systems, such as **CMake** and **Gradle**, which provide more features, better dependency management, and improved scalability for larger projects.

8.7 Phony Targets and Clean-Up

One useful feature in **Makefiles** is the concept of *phony targets*. These are targets that do not generate actual files. Instead, they are used to perform actions like cleaning up intermediate files or running tests. For example:

```

.PHONY: clean
clean:
    rm -f *.o

```

In this case, the **clean** target is a phony target that deletes all object files, ensuring that the project starts from a clean slate.

8.8 Advanced Features of Make

As projects grow, so do the complexities of their build systems. Here are some advanced features of **Make**:

- **Variables:** You can define variables in a **Makefile** to avoid redundancy. For example, you can define the compiler and compiler flags at the beginning of the file and use them throughout the rules.
- **Conditional Execution:** You can use conditionals to check for certain conditions, like whether a file exists or whether a specific variable is set.
- **Pattern Rules:** Make allows the use of pattern rules to generalize build rules for similar files. For example, you can specify that any `.c` file should be compiled with a specific command, eliminating the need to write separate rules for each file.
- **Parallel Execution:** **Make** can be run in parallel, which speeds up the build process by allowing independent tasks to be executed concurrently.

8.9 Building a Custom Build System

Now that we understand the basics of build systems, let's consider how we might build a custom build tool. A custom build tool can be designed to automate tasks like compiling source code, packaging, and running tests. It can be built on top of existing libraries or tools and customized to meet the needs of a specific project.

For example, in Python, we could use a tool like **PyMake** to automate the build process, using a similar approach to **Make** but customized for Python projects. We can also extend the system to include features like dependency resolution, version control, and automated testing.

A key part of building such a tool is understanding the importance of a dependency graph. When defining the dependencies between tasks, we need to ensure that the tasks are executed in the correct order, taking into account which tasks depend on the completion of others.

8.10 Conclusion

In this chapter, we have explored the role of build systems in software development and how they help automate complex tasks like compilation, linking, testing, and deployment. Tools like **Make** have been foundational in the development of modern build systems, but as projects grow, more advanced tools are necessary to handle the increased complexity.

By understanding the core principles of build systems and automation tools, developers can create more efficient, scalable, and maintainable systems. This knowledge is crucial not only for software development but also for managing large-scale projects in a collaborative environment, ensuring that tasks are completed in an organized and efficient manner.

9 From Algorithm to System Design: A Case Study of Google's PageRank

In this chapter, we explore the critical role that algorithms play in system design, particularly focusing on how algorithms, such as the PageRank algorithm, are transitioned into scalable, production-ready systems. This journey is not straightforward, and designing a system to implement an algorithm at scale involves addressing numerous challenges, including optimization, distribution, and ensuring high throughput with low latency. Using Google's search engine as a case study, we examine the key design decisions necessary to transform an algorithmic concept into a fully functioning, large-scale system.

9.1 The Role of Algorithms in System Design

An algorithm provides a clear, step-by-step procedure for solving a problem, but it is just one component of the overall system design. Moving from an algorithm to a real-world system that can handle billions of queries and data points introduces a significant level of complexity. Algorithms must be adapted to handle large datasets, high user traffic, and continuous updates, all while maintaining speed, efficiency, and scalability.

One of the key challenges in system design is understanding how to apply an algorithm to large-scale problems and ensuring that it operates effectively in a distributed environment. This involves choosing the right data structures, ensuring system components interact smoothly, and designing a system that can handle high loads and massive amounts of data.

9.2 The Concept of the Web as a Large Graph

To understand how algorithms such as PageRank work, it is essential to first understand the structure of the web itself. The web can be represented as a very large directed graph where each node represents a web page, and the edges represent hyperlinks between pages. This graph structure is fundamental to understanding how algorithms, such as PageRank, operate, as they assign importance to each page based on the links it receives from other pages.

In the early days of the internet, organizing the web was a significant challenge. Search engines needed to determine how to rank web pages effectively based on their relevance to a user's query. Web directories, such as Yahoo, were initially created by humans to curate and categorize web pages, but this method proved inefficient as the internet grew exponentially. This problem led to the development of automated search engines that could index web pages based on their content and relationships.

9.3 The Problem of Ranking in Web Search

One of the central problems of web search is determining the relevance of a given page in response to a search query. When a user enters a search term, the

search engine needs to return a list of pages that are not only relevant to the query but also trustworthy and authoritative. A search engine must rank pages in a way that highlights the most valuable content for the user.

This ranking problem is not trivial. Early search engines relied on keyword matching, where the frequency of the search term on a page determined its relevance. However, this method was vulnerable to manipulation (e.g., keyword stuffing) and did not account for the quality or authority of the content.

9.4 PageRank: A Solution to the Ranking Problem

In 1998, Larry Page and Sergey Brin, while at Stanford University, developed the PageRank algorithm, which would become the cornerstone of Google's search engine. The core idea behind PageRank is that the importance of a web page is determined by the number and quality of incoming links from other pages. A link from an important page is considered more valuable than a link from a less important page.

The basic concept is simple: each page on the web votes for the pages it links to. The more incoming links a page has from other important pages, the higher its rank. This recursive algorithm assigns a numerical rank to each page, which is based on the ranks of the pages that link to it. This concept allows for a more objective measure of page importance, which can be used to rank search results.

9.5 PageRank Algorithm and Its Mathematical Formulation

PageRank is based on a flow model where each link to a page is considered a vote, and the rank of a page is distributed among its outgoing links. The basic formula for calculating PageRank is as follows:

$$R_j = \sum_{i \in InLinks(j)} \frac{R_i}{OutDegree(i)}$$

Here, R_j represents the rank of page j , $InLinks(j)$ is the set of pages linking to j , and $OutDegree(i)$ is the number of outgoing links on page i . The rank of page j is determined by the sum of the ranks of the pages that link to it, divided by the number of links on each of those pages. This recursive process continues until the ranks converge.

To solve this system of equations, the rank values are initially set to some starting value, and the process is repeated iteratively until the rank values stabilize.

9.6 Challenges in Computing PageRank at Scale

While the PageRank algorithm is conceptually simple, computing it for billions of web pages in real-time presents significant challenges. The sheer size of the

web means that the algorithm needs to handle massive amounts of data. Each page has many incoming and outgoing links, and calculating the rank for each page requires iterating over these links many times.

Furthermore, the computation of PageRank involves solving a system of linear equations, which can be computationally expensive. In practice, the system of equations is solved using methods such as the power iteration method, which iteratively updates the rank values until they converge to a solution. This process can take a long time, especially when dealing with large-scale datasets.

9.7 Optimizing the PageRank Computation

To efficiently compute PageRank at scale, several techniques are used. One approach is to represent the web as a sparse matrix, where the majority of elements are zero (indicating no link between pages). Sparse matrices can be stored and processed more efficiently, reducing the computational cost.

Another optimization is the use of distributed systems to parallelize the computation. Instead of performing the computation on a single machine, the work can be split across many machines, each handling a portion of the web graph. This allows for the computation to be performed much faster, even when dealing with billions of pages.

9.8 Handling Dead Ends and Spider Traps in PageRank

One of the challenges of the PageRank algorithm is handling "dead ends" (pages with no outgoing links) and "spider traps" (pages that form cycles in the graph). In these cases, the algorithm can become stuck, and the rank values may not converge.

To solve these issues, a technique called "teleportation" is introduced. With teleportation, at each step of the iteration, there is a small probability that the algorithm will "jump" to a random page, rather than following the links. This ensures that the algorithm does not get stuck in dead ends or cycles and helps it converge more quickly.

9.9 Designing a System to Implement PageRank

While the PageRank algorithm provides a powerful ranking mechanism, implementing it in a large-scale search engine requires additional considerations. A search engine must be able to handle billions of pages, process queries in real-time, and update the index as new pages are added to the web.

The system must be designed to efficiently crawl the web, process the pages, extract relevant features, and update the PageRank values. This requires a distributed architecture, where different components of the system handle different tasks, such as crawling, indexing, and query processing. The system must also be fault-tolerant, ensuring that it can continue to function even if some components fail.

9.10 The Role of Distributed Systems and MapReduce in Search Engine Design

To handle the massive scale of the web, search engines use distributed systems and technologies like MapReduce. MapReduce allows the computation to be broken down into smaller tasks that can be executed in parallel across many machines. This is particularly useful for tasks like computing PageRank, where the work can be distributed across many nodes in the system.

For example, the crawling process can be parallelized, with different machines crawling different parts of the web. Similarly, the indexing and ranking processes can be distributed, with each machine handling a portion of the web's data.

9.11 Conclusion

The journey from algorithm to system is complex, requiring careful design decisions to ensure that the system operates efficiently at scale. By starting with a concept like PageRank, which provides a clear algorithmic solution to the ranking problem, and translating it into a working system, we can better understand the challenges and considerations involved in system design. The PageRank example illustrates how algorithms must be adapted to handle real-world challenges, such as large-scale data, distributed computation, and system reliability, making it a critical case study in the field of system design.

10 Relational Model and Design of Modern Database Systems

In this chapter, we explore the relational model and its significant impact on the design of modern database systems. The relational model, proposed by Edgar F. Codd in the early 1970s, revolutionized the way data was stored, queried, and manipulated. Through the development of relational algebra and relational calculus, Codd provided a theoretical foundation that allowed databases to become much more flexible and efficient. This chapter will cover the transition from hierarchical and network data models to relational models, the algebraic principles underpinning relational databases, and how modern database management systems (DBMS) are built to support relational data operations.

10.1 The Early Days of Database Systems

Before the advent of relational databases, the management of data was typically done using hierarchical or network data models. These early database systems were tightly coupled to the physical layout of the data on storage devices, and querying the data often required intricate knowledge of the data's structure. Early database systems were designed to run on mainframe computers with high operating costs, where data was primarily stored on magnetic tapes. This sequential access model made it difficult to perform efficient random access to data, and database management systems (DBMS) were primarily built to handle batch processing.

As technology advanced and hard drives were introduced in the 1960s, the need for more efficient data access and management grew. While the early hierarchical and network models were adequate for certain applications, they were rigid and required detailed knowledge of the physical data layout, which hindered the flexibility and ease of use of the systems. It was in this context that Codd, working at IBM in the 1960s, sought to find a better way to manage and query data without the need to rely on the underlying physical data structures.

10.2 Codd's Relational Model and Relational Algebra

In 1970, Codd proposed the relational model as a solution to the limitations of earlier database systems. The relational model introduced the concept of relations, which are essentially tables consisting of rows and columns. The relational model allows for data to be represented at a logical level, freeing users from the need to understand or manipulate the physical data layout.

At the core of the relational model is *relational algebra*, a mathematical system that defines operations on relations. Relational algebra operates on sets of tuples (data records) and defines operations such as selection, projection, and join. These operations can be composed to form complex queries, allowing for powerful and flexible data manipulation.

Codd's relational algebra provided the foundation for what would eventually become SQL (Structured Query Language). The algebraic operations defined

in relational algebra are still present in SQL, though SQL has been extended with additional features to make it more suitable for practical use. The key insight of the relational model was that it provided a high-level abstraction of data, allowing users to query and manipulate data without worrying about its physical storage format.

10.3 Relational Algebra Operations

Relational algebra consists of several basic operations that manipulate relations. These operations include:

- **Selection** (σ): Extracts rows that satisfy a given condition from a relation.
- **Projection** (π): Selects specific columns from a relation.
- **Union** ($\cup$): Combines two relations that share the same schema.
- **Difference** ($-$): Returns the rows in one relation that are not present in another.
- **Cartesian Product** ($\times$): Combines two relations to form all possible pairs of rows.
- **Join** ($\bowtie$): Combines two relations based on a common attribute.
- **Rename** (ρ): Renames the attributes of a relation.
- **Aggregation**: Performs operations such as summing or averaging over a set of tuples (though not part of classical relational algebra, it is often included in extended versions of relational algebra).

These operators form the building blocks for query execution in relational databases. They allow for the creation of complex queries by combining simple operations. For example, a query that retrieves the names of students enrolled in a specific course can be expressed as a combination of selection, projection, and join operations.

10.4 From Relational Algebra to SQL

While relational algebra was a theoretical foundation, it was not directly used by database practitioners. Instead, Codd's work paved the way for SQL, a declarative language that allows users to express queries at a higher level of abstraction. SQL is based on the principles of relational algebra but extends it to provide features such as data manipulation, data definition, and data control.

SQL allows users to specify *what* they want from the database, without needing to specify *how* to retrieve it. The database management system then optimizes the execution plan for the query. For example, a SQL query that retrieves student names from a course may be written as:

```
SELECT name FROM students
JOIN enrollments ON students.id = enrollments.student_id
JOIN courses ON enrollments.course_id = courses.id
WHERE courses.name = 'Database Systems';
```

This SQL query is a higher-level representation of the relational algebra operations needed to retrieve the data. The SQL engine will translate this query into a series of algebraic operations and then execute them on the underlying data.

10.5 Modern Database Systems and Query Optimization

Modern database systems are built around the relational model, and they incorporate advanced query optimization techniques to improve performance. Query optimization is the process of selecting the most efficient execution plan for a query, considering factors such as data size, indexing, and hardware resources.

The query optimization process begins with parsing the SQL query and translating it into a relational algebra expression. The optimizer then generates multiple candidate execution plans, applying transformations to the algebraic expression that preserve its semantics but may improve performance. For example, the optimizer might choose to perform a filter operation before a join, reducing the size of intermediate results.

Once the optimizer has generated a set of possible execution plans, it evaluates them based on cost models, which estimate the resources required for each plan. The optimizer then selects the plan with the lowest estimated cost. This process ensures that queries are executed as efficiently as possible, given the available hardware and resources.

10.6 Relational Algebra as a Theoretical Foundation for Query Execution

The relational algebra provides a solid theoretical foundation for the implementation of database management systems. It allows for clear definitions of data manipulation operations and forms the basis for query optimization techniques. By using algebraic expressions to represent queries, database systems can reason about query execution and optimize performance effectively.

Moreover, relational algebra supports compositionality, meaning that complex queries can be broken down into smaller, simpler components. This modularity makes it easier to understand and optimize database queries, leading to more efficient systems.

10.7 Conclusion

The relational model, as introduced by Codd, revolutionized the way we think about data management. By using relations (tables) and algebraic operations, Codd's work laid the groundwork for modern database systems. The relational

algebra, though theoretical, continues to be a powerful tool for understanding and optimizing query execution.

Today, relational database management systems are the backbone of many applications, from small-scale systems to large enterprise solutions. Through the use of SQL and advanced query optimization techniques, modern DBMSs have been able to provide reliable, scalable, and efficient data management solutions. The principles of relational algebra continue to guide the development and optimization of these systems, ensuring that they remain at the heart of database technology for years to come.

References

- [Bro10] Frederick P. Brooks. *The Design of Design: Essays from a Computer Scientist*. 1st. Addison-Wesley Professional, 2010. ISBN: 0201362988.
- [CK16] Humberto Cervantes and Rick Kazman. *Designing Software Architectures: A Practical Approach*. 1st. Addison-Wesley Professional, 2016. ISBN: 0134390784.
- [FG10] G. Fairbanks and David Garlan. *Just Enough Software Architecture: A Risk-Driven Approach*. Jan. 2010.
- [Gam+95] Erich Gamma et al. *Design patterns: elements of reusable object-oriented software*. USA: Addison-Wesley Longman Publishing Co., Inc., 1995. ISBN: 0201633612.
- [Low19] Juval Lowy. *Righting Software: A Method for System and Project Design*. 1st. Addison-Wesley Professional, 2019. ISBN: 0136524036.
- [Ous18] John Ousterhout. *A Philosophy of Software Design*. 1st. 2018. ISBN: 1732102201.